AI-BASED SMART PATIENT REFERRAL AND HOSPITAL SELECTION SYSTEM

A PROJECT REPORT

Submitted by

DHANALAKSHMI C **241901023**

BHARATH V **241901018**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)



RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report "AI – BASED SMART PATIENT REFERRAL AND HOSPITAL SELECTION SYSTEM" is the bonafide work of "BHARATH V (2116241901018), DHANALAKSHMI C (2116241901023)" who carried out the project work under my supervision.

SIGNATURE

Dr. Bhuvana S, Ph.D.,
ASSISTANT PROFESSOR(SG)
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

Referring a patient to a suitable hospital within a limited time window is difficult. Hospitals differ in specialties, treatment facilities and resource availability, and manual referral is slow and inconsistent. Machine-learning systems can help, but they behave as black boxes and need large training datasets. This paper presents an AI-Based Smart Patient Referral and Hospital Selection System, a web-based prototype that uses explainable rule-based reasoning instead. A knowledge base of weighted symptom rules is applied by forward chaining, from the symptoms entered to a possible specialty area, a severity level and red-flag emergency patterns, with a trace explaining each inference. Hospitals are then evaluated against the inferred need. Hard constraints exclude disabled hospitals and block referrals to hospitals marked unavailable. A transparent 0–100 score then combines specialty match, resources, emergency capability, distance (Haversine) and specialist availability, with best-first ordering of the results. A caretaker confirms the recommendation, and the referral moves through a tracked workflow (confirmed, submitted, accepted or rejected with a reason, completed) with a stored status history. Hospitals update their status and resources, and rankings change immediately. The system uses Node.js, Express and SQLite with role-based interfaces for patients, caretakers, hospitals and administrators, and synthetic data only. Nine scenario tests check the rule logic and workflow. It provides decision support, not diagnosis, and is not clinically validated.

Keywords: knowledge-based system, forward chaining, explainable AI, clinical decision support, hospital selection, patient referral

ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for enabling us to complete this project successfully. We express our sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his dedicated efforts in providing quality education at this esteemed institution. We are deeply grateful to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her constant motivation and encouragement, which inspired us throughout the completion of this project. We also thank our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We express our sincere gratitude to our College Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement.

We are especially thankful to our project guide, Dr. Bhuvana S., Ph.D., Assistant Professor(SG), for her valuable guidance, continuous encouragement, and support throughout the project. Finally, we thank our parents and friends for their direct and indirect contributions, encouragement, and support in the successful completion of this project.

DHANALAKSHMI C (2116241901023)

BHARATH V (2116241901018)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	
	1.2 PROBLEM STATEMENT	
	1.3 OBJECTIVES	
	1.4 SCOPE	
	1.5 SECURITY REQUIREMENTS/ SECURITY CONSIDERATIONS	
	1.6 SDG ALIGNMENT	
2	**LITERATURE REVIEW**	
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	
	2.2 RESEARCH GAPS	
	2.3 PROPOSED SYSTEM	
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	
3	**SYSTEM DESIGN**	
	3.1 SYSTEM ARCHITECTURE	
	3.2 MODULE DESIGN	
	3.3 DATA FLOW/UML DIAGRAMS	
	3.4 THREAT MODEL AND ATTACK SURFACE	
	3.5 SECURITY CONTROLS	

4 SYSTEM DESCRIPTION

4.1 DATA COLLECTION

4.2 DATA PREPROCESSING

4.2.1 ALGORITHMS / METHODOLOGY

4.2.2 RESULTS AND DISCUSSION

5 CONCLUSION

REFERENCES

APPENDIX

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG ALIGNMENT OF THE PROPOSED PROJECT	
2.1	COMPARISON OF EXISTING AND PROPOSED SYSTEMS	
3.1	SYSTEM LAYERS AND RESPONSIBILITIES	
3.2	HOSPITAL RECOMMENDATION SCORING CRITERIA	
3.3	SYSTEM ATTACK SURFACE	
4.1	DATA SOURCES OF THE PROPOSED SYSTEM	
4.2	SAMPLE HOSPITAL DATA	
A.1	DEMONSTRATION USER ACCOUNTS	
E.1	HARDWARE AND SOFTWARE REQUIREMENTS	

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
2.1	FLOW OF THE SYSTEM	
3.1	SYSTEM ARCHITECTURE	
3.2	DFD FOR PATIENT DATA COLLECTION	
3.3	DFD FOR PATIENT REQUIREMENT ANALYSIS	
3.4	DFD FOR HOSPITAL EVALUATION AND RANKING	
3.5	DFD FOR REFERRAL RECOMMENDATION AND EXPLANATION	
3.6	DFD FOR HOSPITAL ADMIN AND DATABASE	
3.7	DATA FLOW DIAGRAM FOR SYSTEM	
3.8	USE CASE DIAGRAM	
3.9	SEQUENCE DIAGRAM	
3.10	REFERRALL STATE DIAGRAM	
3.11	ENTITY-RELATIONSHIP DIAGRAM	
4.1	AI ANALYSIS RESULT SCREEN	
4.2	HOSPITAL RANKING WITH SCORE BREAKDOWN	
4.3	ROUTE MAP WITH ROUTE OPTIONS	
4.4	SCENARIO TEST RESULTS (9 OF 9)	

CHAPTER 1
INTRODUCTION

Healthcare delivery depends heavily on timely and appropriate referral decisions, made at the moment a patient's care is handed to a hospital that is equipped to treat their condition. A referral that is made too slowly, or sent to a hospital that later proves unable to provide the required specialist, facility or critical-care support, costs the patient time. In emergencies that lost time can directly affect clinical outcomes.

As hospital networks have grown, so has the variation between hospitals in the specialties, facilities and resources they offer. In principle this diversity is beneficial, because specialised care exists somewhere in the network. In practice it becomes a liability when there is no systematic way to match a patient's specific needs against this fragmented landscape in real time. Healthcare workers and caretakers are often left to rely on personal familiarity, outdated directories, or phone calls made under time pressure.

Computer support for this problem has been studied, including machine-learning approaches to hospital referral and triage [1], [2]. Such systems usually need large amounts of training data and often behave as black boxes, which makes them hard to trust and validate in critical healthcare settings. Studies of automated symptom tools also show that their advice can be unreliable [5], so any such system must be transparent and must keep a human in charge of the final decision.

This project, the AI-Based Smart Patient Referral and Hospital Selection System, addresses this gap using classical, symbolic Artificial Intelligence rather than Machine Learning. It treats referral as a reasoning and matching problem. A rule-based knowledge base of weighted symptom rules, applied by Forward Chaining, infers a possible specialty area and an urgency level from the patient's symptoms, existing conditions and emergency level. Constraints exclude disabled hospitals and block referrals to unavailable ones. The remaining hospitals are ranked, in best-first order, by a transparent score based on specialty match, resources, emergency capability, distance and specialist availability. Every recommendation is accompanied by an explicit explanation, namely the symptoms that triggered each rule and the score breakdown for each hospital.

The system is a web-based prototype with separate interfaces for patients, caretakers, hospitals and administrators. A caretaker confirms each recommendation, and the referral is tracked until the hospital accepts it, rejects it with a stated reason, or completes it. It uses synthetic data only and provides decision support, not medical diagnosis.

1.1 BACKGROUND

Referring a patient from initial contact to a hospital capable of providing definitive treatment is one of the less digitized coordination problems in healthcare. Diagnosis, treatment planning, and hospital record-keeping have all seen substantial software investment, but the referral decision itself is still overwhelmingly made by a person using judgment and whatever fragmented information is available at that moment, not for lack of data but because each hospital's specialty, facility, and availability information is rarely accessible in a single, real-time, decision-ready form. Recent AI research has leaned heavily toward Machine Learning for healthcare tasks such as medical imaging and risk prediction, but these approaches are less suited to referral, where the core need is transparent, rule-governed matching rather than pattern recognition from large historical datasets. Rule-based expert systems have a long history in medical decision support, dating to early systems such as MYCIN, an expert system developed at Stanford University in the early 1970s to advise on severe bacterial infections and antibiotic treatment using IF-THEN rules. Such systems remain attractive because they keep every decision traceable to an explicit rule. This project builds on that tradition, combining rule-based inference with constraint-based filtering and heuristic ranking of hospitals.

1.2 PROBLEM STATEMENT

Healthcare providers often lack a centralized, real-time decision-support system capable of matching a patient's clinical needs against the current capabilities of hospitals in a network. This produces three recurring problems: inconsistency, where different staff reach different conclusions for similar cases; delay, where confirming a hospital's current specialist or bed availability takes time the patient may not have; and mismatch, where a patient is referred to a hospital that later proves to lack an essential requirement, forcing a second, costlier referral. These problems are most serious in emergencies, where the patient must reach a suitable hospital within a limited time window, and hospital specialties, facilities and resources change during the day, so a directory that was correct in the morning may be wrong by the evening. Machine-learning decision systems could help, but they typically need large volumes of training data and behave as black boxes, which makes them difficult to trust and validate in critical healthcare settings. What is needed is a system that determines the medical specialty and urgency a patient needs, evaluates current hospital data against those needs, explains why its recommendation was made, keeps a human in charge of the final decision, and records the referral from confirmation to outcome.

1.3 OBJECTIVES

The main objective is to design and build a working prototype of an explainable decision-support system that recommends a suitable hospital for a patient and records the referral through to its outcome.

The specific objectives are:

- To collect patient details, symptoms, existing conditions and location through role-based web interfaces for patients, caretakers, hospitals and administrators.
- To analyse symptoms with an explainable rule-based knowledge base, using forward chaining to suggest a possible specialty area, a severity level and red-flag emergency warnings, without naming any disease.
- To exclude disabled hospitals and block referrals to unavailable ones, and to rank the remaining hospitals with a transparent project-specific score from 0 to 100, based on specialty match, resources, emergency capability, distance and specialist availability.
- To explain every recommendation, showing the symptoms that triggered each rule and the score breakdown for each hospital.
- To support a caretaker-confirmed referral workflow (Pending, Caretaker Confirmed, Submitted, Hospital Accepted or Rejected with a reason, Completed) with a stored status history.
- To let hospitals update their emergency status, resources, specialists and facilities, so that later rankings change immediately.
- To provide administrator tools for managing hospitals and users, with reports and an activity log.
- To show the patient's location and a route to the hospital on a map, using predefined Chennai areas.
- To verify the rule logic and workflow through documented software scenario tests.

1.4 SCOPE

1.4.1 In scope

The proposed system is a full-stack web application developed using HTML, CSS, and JavaScript for the front end, with Node.js, Express, and SQLite for the back end, featuring four user roles with role-based access control. It provides rule-based analysis across five medical specialties: Cardiology, Neurology, Pulmonology, Orthopedics, and General Medicine, and recommends suitable hospitals from five demonstration hospitals in Chennai, with administrator tools to add and manage additional hospitals. The application supports the complete referral workflow, including creation, confirmation, submission, acceptance, rejection, and completion, along with referral history and reports. It also includes patient registration and login, registered phone number management, and an optional emergency contact with a notification-consent setting that is stored for reference only. Additionally, the system integrates a route map showing the route between the patient's area and the recommended hospital.

1.4.2 Out of scope

The system is designed for decision support only and does not provide medical diagnoses or make treatment decisions. It does not include machine-learning models, large clinical datasets, or clinical validation, and it is not integrated with real hospitals, electronic health records, ambulance dispatch services, or any production healthcare system. All data used in the prototype is synthetic, and the system does not send SMS messages or other alerts. Exact street-address lookup is not supported; instead, locations are matched to predefined areas in Chennai, and hospital ranking uses straight-line distance estimates. Furthermore, the project does not include a dedicated mobile application or claim compliance with healthcare regulations such as HIPAA.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

The system handles personal and health-related information, so security was considered from the start. The prototype uses synthetic data only and is not production healthcare security.

1.5.1 Security requirements

- **Authentication:** Only registered users may use the system, and each signs in with a username and password.
- **Password protection:** Passwords must never be stored or shown as plain text.

- **Authorization:** Each role may access only what it needs. Patients see only their own record, caretakers see their own patients and referrals, hospitals see only referrals submitted to them, and administrators manage the system.
- **Data integrity:** Inputs must be validated, and referral status changes must follow the allowed workflow.
- **Accountability:** Important actions must be recorded.
- **Privacy:** A patient's registered contact number must be protected from unwanted changes.

1.5.2 Measures implemented

Passwords are securely stored as bcrypt hashes. Every API request is verified on the server for authentication and role-based authorization, ensuring that security does not rely solely on hiding interface elements. Disabled accounts lose access immediately because their status is checked on every request. Session cookies are configured as HTTP-only with a SameSite restriction, and the session secret is stored in an environment file rather than in the source code. All database queries use parameterized statements to prevent SQL injection. User input is validated on the server for type, length, and permitted values, while displayed text is escaped to reduce the risk of cross-site scripting (XSS). Referral transitions are validated against permitted workflow steps, user roles, and ownership, while duplicate active referrals and referrals to unavailable or disabled hospitals are blocked. The registered phone number cannot be changed through the profile-update API. Additionally, an activity log records important actions, including logins, referral changes, and administrative operations, to support monitoring and accountability.

1.5.2 Known limitations

The prototype currently runs over plain HTTP in a local environment, so HTTPS would be required for a real deployment to secure data transmitted between clients and the server. It does not include login-attempt limits, multi-factor authentication, or email and OTP verification. Sessions are stored in memory and are reset whenever the server restarts. Demo passwords are included in the seed file because the data is synthetic and intended for testing purposes. The map functionality relies on public CDNs, mapping services, and routing services, which require an internet connection. Before real-world deployment, the system would require legal and regulatory review, comprehensive audit trails, reliable backups, and encryption of stored data. The prototype does not claim HIPAA compliance.

1.6 SDG ALIGNMENT

S.No	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	**SDG 3** – Good Health and Well-being	**Target 3.8:** achieving universal health coverage, including access to quality essential health-care services for all.	Helps patients and caretakers reach a suitable hospital with less guesswork and delay. The system suggests a possible specialty area, ranks hospitals by a transparent score using live availability, and shows red-flag warnings that tell users to seek professional care at once. It supports decisions and does not diagnose.
2	**SDG 9** – Industry, Innovation and Infrastructure	**Target 9.5:** strengthening scientific research and upgrading technological capabilities.	Applies explainable rule-based AI and web technology to a real service problem. It builds a working full-stack prototype (Node.js, Express and SQLite) that combines symptom analysis, hospital ranking, referral tracking and a route map, and it can be extended with real data later.
3.	**SDG 16** – Peace, Justice and Strong Institutions	**Target 16.6:** building institutions that are effective, accountable and transparent at every level.	Makes the referral process transparent and accountable. Every recommendation is explained, every status change is stored with who made it and when, hospital rejections require a stated reason, and an activity log records key actions.

Table 1.1 SDG Alignment of the proposed project

CHAPTER 2
LITERATURE REVIEW

This chapter reviews ten sources covering rule-based clinical decision support, hospital and provider selection, referral triage, symptom-based assessment, and the measurement of geographic access to care. It then identifies the gaps these sources leave and relates them to the proposed system. The reviews are based on the published abstracts and key findings of each work, not on an exhaustive survey. The reference numbers [1] – [10] are listed at the end of the report.

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

2.1.1 Conventional referral

In most settings, referral is still manual. A caretaker or staff member chooses a hospital using personal experience, printed directories or phone calls. There is no common view of hospital availability, no recorded reason for the choice, and no tracking after the patient is sent. This is common practice rather than the finding of any single study, and it motivates the need for a transparent, trackable alternative.

2.1.2 Rule-based and knowledge-based decision support

The idea of encoding medical reasoning as explicit rules is long established. The classic rule-based expert system MYCIN [9] used a knowledge base of several hundred production (IF–THEN) rules together with certainty factors to recommend antimicrobial therapy, and it could explain its reasoning by tracing the rules it fired. This explainability is the main attraction of rule-based systems for a student prototype.

Sutton et al. [6], in a widely cited overview of clinical decision support systems, distinguish knowledge-based systems, which apply explicit IF–THEN rules, from non-knowledge-based systems, which learn from data using machine learning. They summarise the benefits of such systems, including better adherence to guidelines and improved safety, as well as the risks, such as alert fatigue and disruption of clinical workflow, and they set out strategies for successful adoption. Their framing supports the choice of a transparent, knowledge-based design for this project, where no large labelled dataset is available.

2.1.3 Hospital and provider selection

Chi, Street and Ward [1] propose a referral expert system that has no explicit knowledge base; it builds machine-learning classifiers from a collection of labelled cases and uses an optimisation step to recommend the hospital that maximises the probability of a desired outcome. The decision considers institutional characteristics, patient risks, travelling distance, and the chances of survival and complications. This demonstrates multi-factor hospital selection that includes distance, but it depends on labelled training data and the reasoning is harder to trace.

Omotosho et al. [4] adapt a provider-selection model that uses a multi-attribute decision function, choosing the best provider from patient and provider factors across fourteen referral criteria. This is the closest match to the multi-criteria hospital ranking used in the proposed system, though it does not address symptom analysis or live availability.

2.1.4 Referral triage and symptom-based assessment

Abdel-Hafez et al. [2] used machine learning to sort referral letters into urgency categories. Analysing 17,378 ENT referrals from two Queensland hospitals, they found 53.8% agreement between the existing categories and the model's predictions, showing that AI can support referral triage but with only moderate agreement.

Määttä et al. [3] tested a clinical decision support system in a university-hospital emergency department, where patients completed a web-based form while waiting for the triage nurse. The system was equally sensitive to physicians in urgent cases, with acceptable differential-diagnosis accuracy and good usability. This shows the value of patient-entered data and urgency detection, although it was evaluated within a clinical setting.

Two studies examine symptom checkers directly. Semigran et al. [5] audited 23 online symptom checkers and found that they gave the correct diagnosis first in 34% of cases and appropriate triage advice in 57%, concluding that the tools have deficits in both triage and diagnosis. Chambers et al. [7], in a systematic review of digital and online symptom checkers and triage services for urgent health problems commissioned for a national health service, reported a mixed picture: the tools were often cautious or risk-averse in their triage advice, the overall evidence base was limited, and there was little high-quality evidence either way on patient safety. Together, [5] and [7] show why patient-facing tools need red-flag warnings, clear disclaimers and a human confirming the decision.

2.1.5 Geographic accessibility and distance measurement

Because distance to care is one factor in hospital ranking, two further sources inform that component. Luo and Wang [8] synthesise GIS-based measures of spatial accessibility to health care and apply them in a case study of the Chicago region, showing that geographic access is a legitimate, quantifiable factor in where patients can realistically be treated. For the underlying computation, the proposed system uses the haversine formula [10], the standard method for the great-circle distance between two latitude–longitude points, to estimate straight-line distance from the patient's area to each hospital.

2.1.6 Security approaches

The abstracts of the reviewed studies concentrate on decision quality and usability and do not describe their security mechanisms. For security, this chapter therefore relies on the standard safeguards that health regulations expect. Under the US HIPAA Security Rule, technical safeguards cover five areas: access control, audit controls, integrity protection, person-or-entity authentication, and transmission security. In practice, health information systems usually implement these through unique user accounts, role-based access, password protection, activity logging, validation to prevent unauthorised changes, and encrypted connections. These are the approaches against which the proposed system's security (Section 1.5) is compared. The prototype uses a basic subset of them and makes no claim of compliance.

2.2 RESEARCH GAPS

These gaps come from the five studies above, based on their published abstracts. They are not the result of an exhaustive survey.

- **Data dependence and limited transparency.** The machine-learning approaches [1], [2] learn from labelled cases and, in [1], work without an explicit knowledge base, which makes the reasoning harder to trace. As the CDSS overview notes [6], knowledge-based systems offer a transparent alternative, which is needed where large datasets are not available.
- **A fragmented scope.** The studies address triage or diagnosis [2], [3], [5], [7], or provider selection [1], [4], but none describes a single system that goes from symptoms to the needed specialty, then to hospital ranking, a confirmed referral and a tracked outcome.

- **Reliability and the role of the human.** The audits and reviews of symptom checkers [5], [7] found deficits and over-caution, so patient-facing tools need red-flag warnings, clear disclaimers and a human confirming the decision. Even a well-performing system [3] was assessed within a clinical setting.
- **Live hospital availability.** The reviewed sources consider several hospital factors, and geographic access can be measured [8], but none describes hospitals updating their own status and resources so that the next ranking changes at once.
- **Accountability and tracking.** The abstracts do not describe a stored status history, stated reasons for rejection, or an activity log for referrals.
- **Security by design in prototypes.** Security is not discussed in the abstracts, and decision-support prototypes often leave it out. A prototype should at least demonstrate authentication, role-based access, hashed passwords, validation and an audit trail.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed AI-Based Smart Patient Referral and Hospital Selection System is a web-based prototype that links symptom analysis, hospital ranking and referral tracking. It uses classical rule-based AI and no machine learning. It serves four roles: patient, caretaker, hospital and administrator.

The main techniques are:

The system uses a knowledge base of weighted symptom rules applied through forward chaining, with synonym matching and red-flag warnings to identify potential emergencies. It applies constraint-based filtering to exclude disabled hospitals and block referrals to unavailable hospitals. Eligible hospitals are ranked using a transparent score out of 100, based on specialty match (35 points), resource availability (20 points), emergency capability (15 points), distance (15 points), and specialist availability (15 points), with penalties applied to hospitals marked as Limited or Unavailable. Hospitals are then ordered from highest to lowest score. The Haversine formula is used to calculate distances between predefined areas of Chennai. Additionally, the system implements a referral workflow with the statuses Pending, Caretaker Confirmed, Submitted, Hospital Accepted or Rejected, and Completed, while recording every status change for tracking and accountability. The system is built with HTML, CSS and JavaScript on the front end, and Node.js, Express and SQLite on the back end. It uses synthetic data only and provides decision support, not diagnosis.

The flow of the system is:

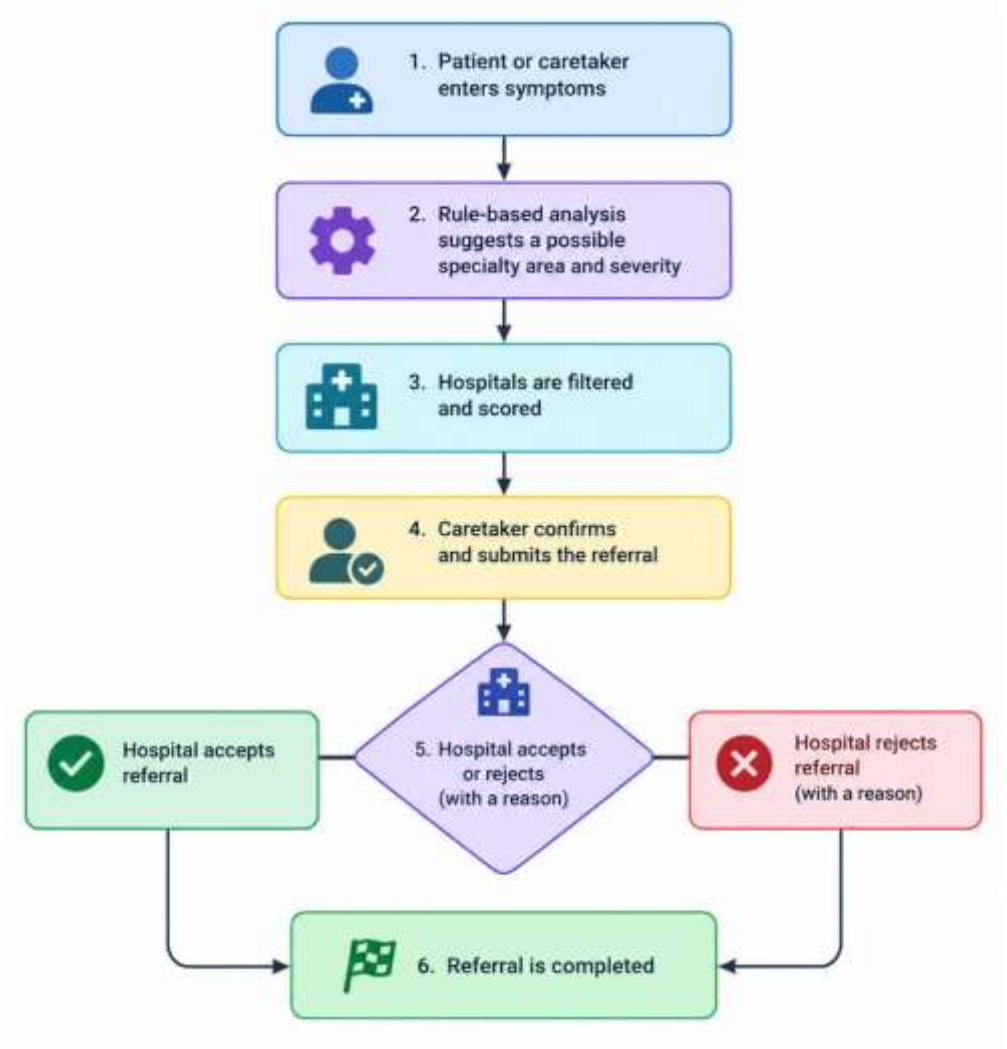


Fig 2.1 Flow of the system

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

- To suggest a possible medical specialty and urgency level from symptoms using transparent rules.
- To rank suitable hospitals with a score that anyone can inspect and understand.
- To explain each recommendation through the symptom trace and the score breakdown.
- To keep a human in charge, with the caretaker confirming every referral.
- To reflect live hospital status and resources in every new ranking.
- To track each referral from creation to outcome, with stored history and stated rejection reasons.
- To protect data with login, role-based access, hashed passwords, validation and an activity log.

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

The proposed system offers several practical advantages over manual referral and black-box approaches. Its results are explainable, because every recommendation shows the symptoms that triggered each rule and the score components behind each hospital's ranking. It needs no training data, since the rules work without a large dataset and can be edited directly when requirements change. Hospitals can update their status, resources and specialists, and the next ranking reflects those changes immediately, so recommendations follow current availability. One system covers the complete workflow of analysis, ranking, confirmation, hospital response and history, and every status change is stored with who made it and when, while a rejection must carry a reason, which gives accountability. Red-flag warnings, an "insufficient information" result, clear disclaimers and caretaker confirmation keep the system in a supporting role and leave the final decision with a human. Security is considered in the design through server-side role-based access, hashed passwords, parameterised queries and an activity log. Finally, the route map falls back to an offline diagram if the internet or the map service is unavailable, so the demonstration does not depend on a network connection.

Comparison Criteria	Conventional Referral	Machine-learning systems	Proposed System
Method of hospital selection	Phone calls and guesswork	Model prediction	Transparent score out of 100
Explanation of the result	None	Often limited	Symptom trace and score breakdown
Training data requirement	Not applicable	Large labelled dataset needed	Not needed (rule-based)
Use of current hospital availability	Unknown or outdated	Not typically included	Hospitals update their own status and resources
Referral tracking	None	Not typically included	Full status history with reasons

Table 2.1 Comparison of Existing and Proposed Systems

Table 2.1 Compares conventional manual referral, machine-learning-based referral systems and the proposed system across five criteria: how the hospital is selected, whether the result is explained, whether training data is required, whether current hospital availability is used, and whether the referral is tracked. Manual referral depends on phone calls and personal judgment, gives no explanation, and has no view of current availability or record of the outcome. Machine-learning systems can predict a suitable hospital from past cases, but they need large

labelled datasets, often explain their results only to a limited degree, and are not typically designed to use live availability or to track the referral. The proposed system selects hospitals with a transparent score, shows the symptoms and score components behind every result, needs no training data, reflects hospital updates in the next ranking, and stores the full status history of each referral. The table describes how each approach is designed to work and is not a measured performance comparison.

CHAPTER 3

SYSTEM DESIGN

I made seven diagrams, one for each figure referenced in this chapter, and attached them. This chapter describes only what the prototype actually does.

3.1 SYSTEM ARCHITECTURE

The system uses a three-layer, client-server design. The browser talks to a Node.js and Express server through a REST API that exchanges JSON, and the server stores data in a SQLite database.

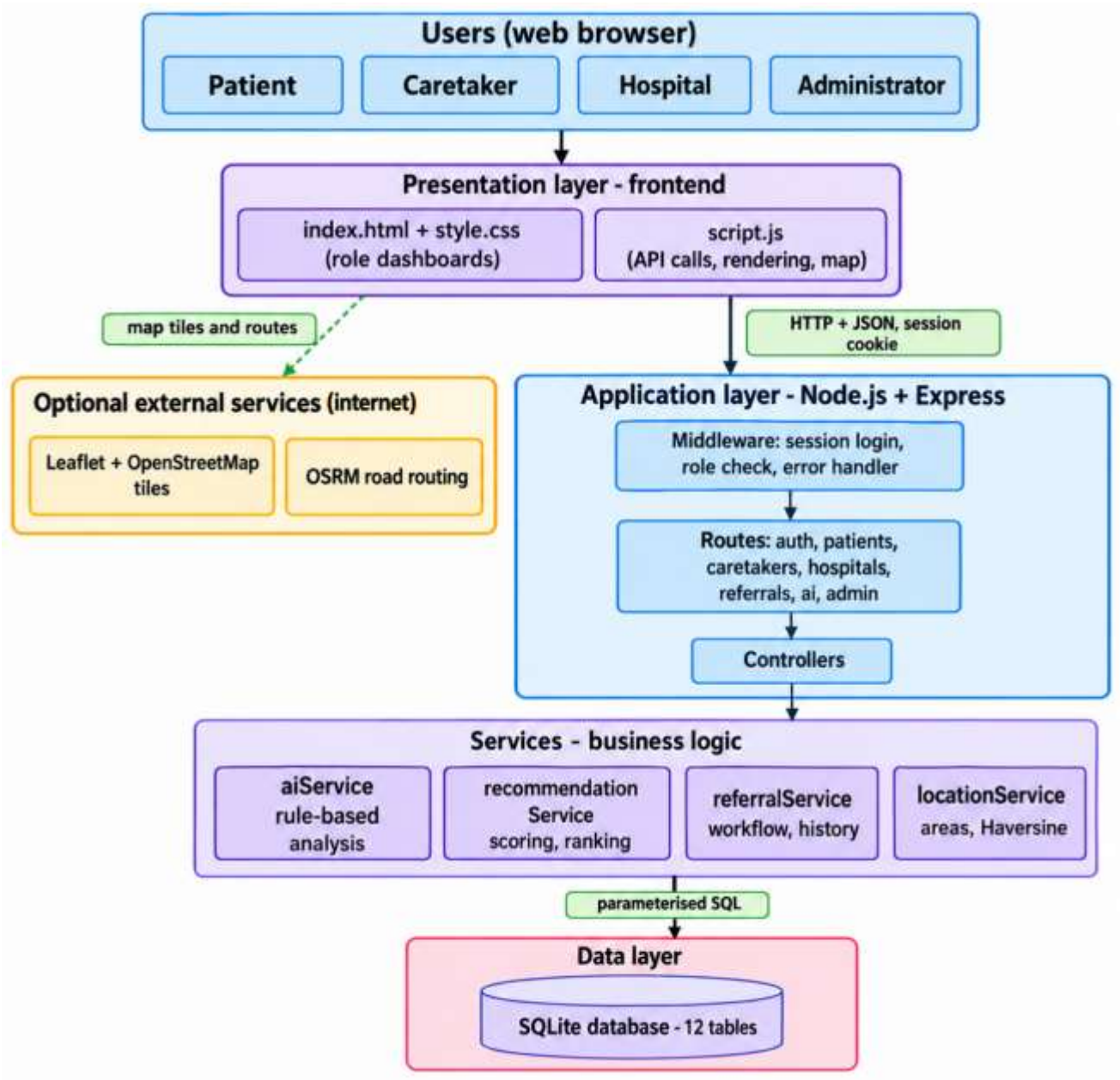


Fig 3.1 System architecture

Layer	Components	Responsibility
Presentation	index.html, style.css, script.js	Single-page interface with a dashboard for each role, forms, tables and the route map. It calls the API and displays results.
Application	Express server, middleware, routes, controllers, services	Authentication, role checks, validation and business logic.
Data	SQLite via better-sqlite3	Stores users, patients, hospitals, referrals, analyses and logs in 12 tables.
External(optional)	Leaflet, OpenStreetMap tiles, OSRM routing	Real street map and road routes. If unreachable, the app shows an offline diagram.

Table 3.1 System Layers and Responsibilities

Table 3.1 Summarises the four layers of the system. The presentation layer handles the user interface, the application layer holds the server logic and security checks, the data layer stores all records in a SQLite database, and an optional external layer supplies the map and road routes when an internet connection is available.

3.1.1 Application layer structure

- Middleware (authMiddleware.js) checks the session login, the user's role, and handles errors.
- Routes (auth, patients, caretakers, hospitals, referrals, ai, admin) map each URL to a controller.
- Controllers read the request, call a service and send the response.
- Services hold the business logic: aiService (rule-based analysis), recommendationService (hospital scoring and ranking), referralService (workflow and history) and locationService (Chennai areas and Haversine distance).

3.1.2 Request Processing Flow

Each request from the browser is sent to the server through the fetch interface, to an address beginning with /api/, together with the session cookie. The server first checks that the user is logged in and then checks that the user's role is allowed to perform the action. The request is then passed to the relevant controller, which calls the required service. The service reads or writes the database using parameterised SQL queries, and the server returns the result to the browser as a JSON response.

3.1.3 Technology stack:

HTML5, CSS3 and vanilla JavaScript (front end); Node.js 22 and Express (back end); SQLite with better-sqlite3 (database); bcryptjs and express-session (security); dotenv for configuration.

3.2 MODULE DESIGN

The system has five modules.

3.2.1 PATIENT DATA COLLECTION

- **Purpose:** Register users and collect the patient's details.
- **Inputs:** Name, phone and password at registration; age, gender, hours of pain, emergency level, area, symptoms (ticked or typed), existing conditions, medications, optional emergency contact and notification consent.
- **Processing:** Server-side validation, bcrypt hashing, session creation. The patient's name becomes the username, and the phone number is stored separately and is read-only.
- **Output:** A patient record.
- **Data and code:** Tables users and patients; authController, patientController. API: /api/auth/register, /api/auth/login, /api/patients.

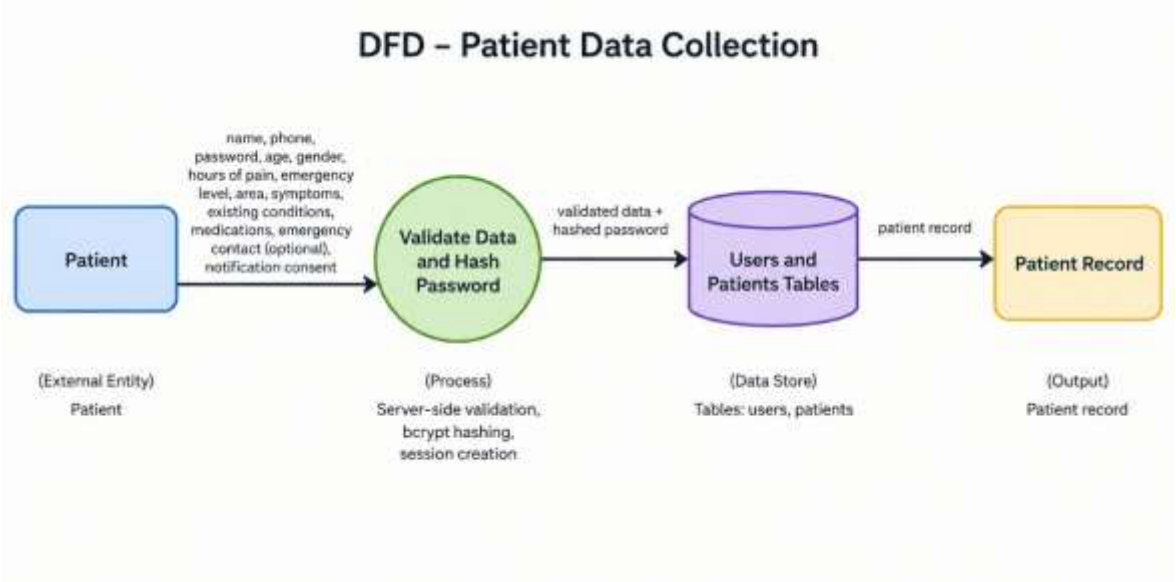


Fig 3.2 Data Flow Diagram (DFD) for Patient Data Collection

3.2.2 PATIENT REQUIREMENT ANALYSIS (RULE-BASED AI)

- **Purpose:** Turn symptoms into a possible specialty area and urgency level, with an explanation.
- **Processing (forward chaining over a rule base):**
    1. Normalise the text.
    2. Detect known symptoms, including synonyms such as “heart pain” and negation such as “no chest pain”.
    3. Add the weights of detected symptoms for each specialty.
    4. Pick the highest score. In case of a tie, use the order: Cardiology, Neurology, Pulmonology, Orthopedics, General Medicine.
    5. Apply red-flag rules and set the severity. The entered emergency level can raise severity but never lower it.
    6. Build the explanation and a trace.
- **Output:** Specialty, severity, confidence, detected symptoms, scores, red flags, warning, reason and trace. If no symptom is recognised, the result is “Insufficient information”.
- **Data and code:** Table symptom_analyses; aiService. API: POST /api/ai/analyze.

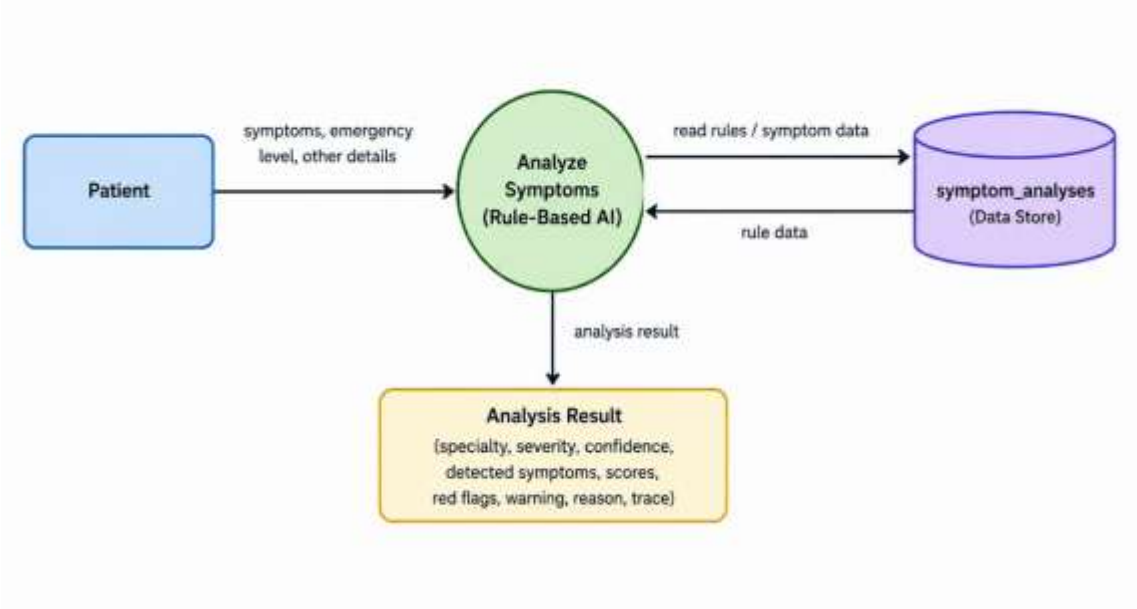


Fig 3.3 Data Flow Diagram (DFD) for Patient Requirement Analysis (Rule-Based AI)

3.2.3 HOSPITAL EVALUATION AND RANKING

- **Purpose:** Compare hospitals and rank them with a transparent score.
- **Processing:** Disabled hospitals are left out. Each remaining hospital gets a project-specific recommendation score from 0 to 100, and hospitals are sorted best first.
- **Output:** A ranked list with the score breakdown, distance, matched specialty, facilities, emergency status, resources, specialists and an explanation.
- **Data and code:** Recommendation Service, location Service. API: POST /api/ai/recommend.

Component	Maximum	Rule
Specialty match	35	35 for the exact specialty, 10 for a General Medicine fallback, 0 otherwise
Resources	20	resources (0–100) scaled to 20
Emergency capability	15	24/7 Emergency 12 or Emergency 9, plus 3 for an Emergency Medicine specialty, plus 3 for an ICU, capped at 15
Distance	15	15 at 0 km, falling in a straight line to 0 at 30 km (Haversine)
Specialist availability	15	up to 10 specialists give the full score
Penalty	-	Unavailable: 40 (urgent) or 25 (routine). Limited: 10 or 5.

Table 3.2 Hospital recommendation scoring criteria

The hospital recommendation scoring criteria shown in Table 3.2 are used to evaluate and rank hospitals based on specialty match, available resources, emergency capability, distance, and specialist availability, with penalties applied for unavailable or limited services.

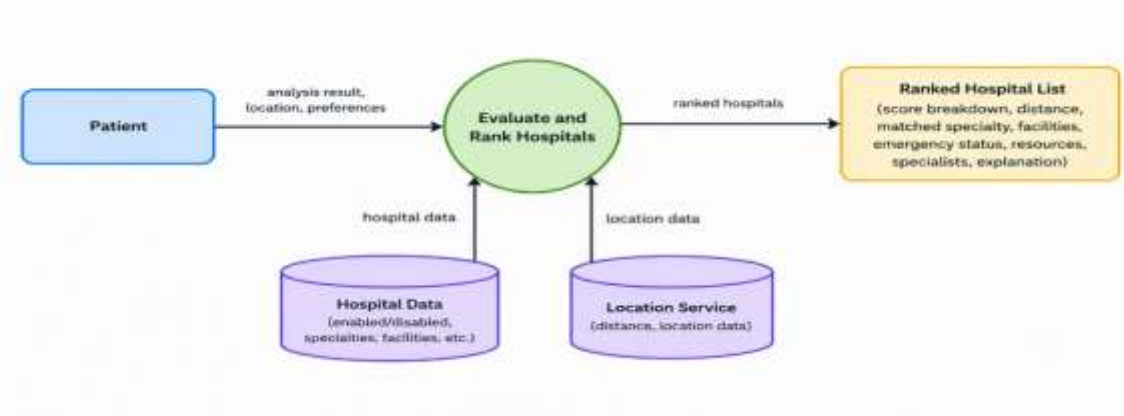


Fig 3.4 DFD for Hospital Evaluation and Ranking

3.2.4 REFERRAL RECOMMENDATION AND EXPLANATION

- **Purpose:** Turn a chosen hospital into a tracked referral.
- **Processing:** A caretaker creates the referral. The server runs the analysis and ranking again so the stored score can't be forged. It blocks duplicates and unavailable or disabled hospitals. Every status change is validated for the allowed step, the allowed role and ownership, and is stored in a history table.
- **Output:** A referral with its AI score, explanation, status and history.
- **Data and code:** Tables referrals and referral_status_history; referralService. API: /api/referrals.

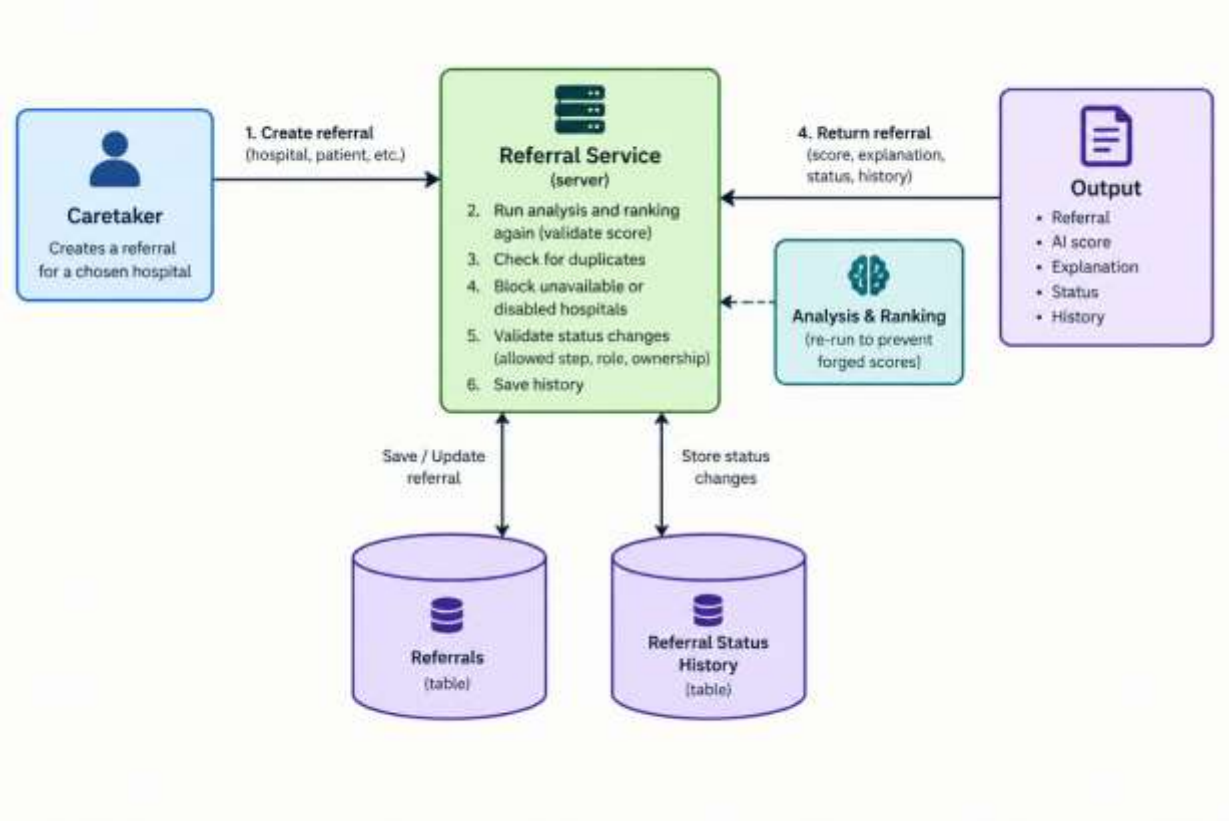


Fig 3.5 DFD for Referral Recommendation and Explanation

3.2.5 HOSPITAL ADMINISTRATION AND DATABASE MANAGEMENT

- **Purpose:** Keep hospital data current and manage the system.
- **Functions:** Hospitals update their emergency status, resources, specialists and facilities, and the next ranking reflects it. Administrators add, edit, enable and disable hospitals and users, view reports and the activity log, and run the nine scenario tests.
- **Database:** Created and seeded automatically on first start, with a migration for added columns.
- **Data and code:** Tables hospitals, hospital_specialties, hospital_facilities, hospital_resources, activity_logs; hospitalController, adminController, initDatabase. API: /api/hospitals, /api/admin/....

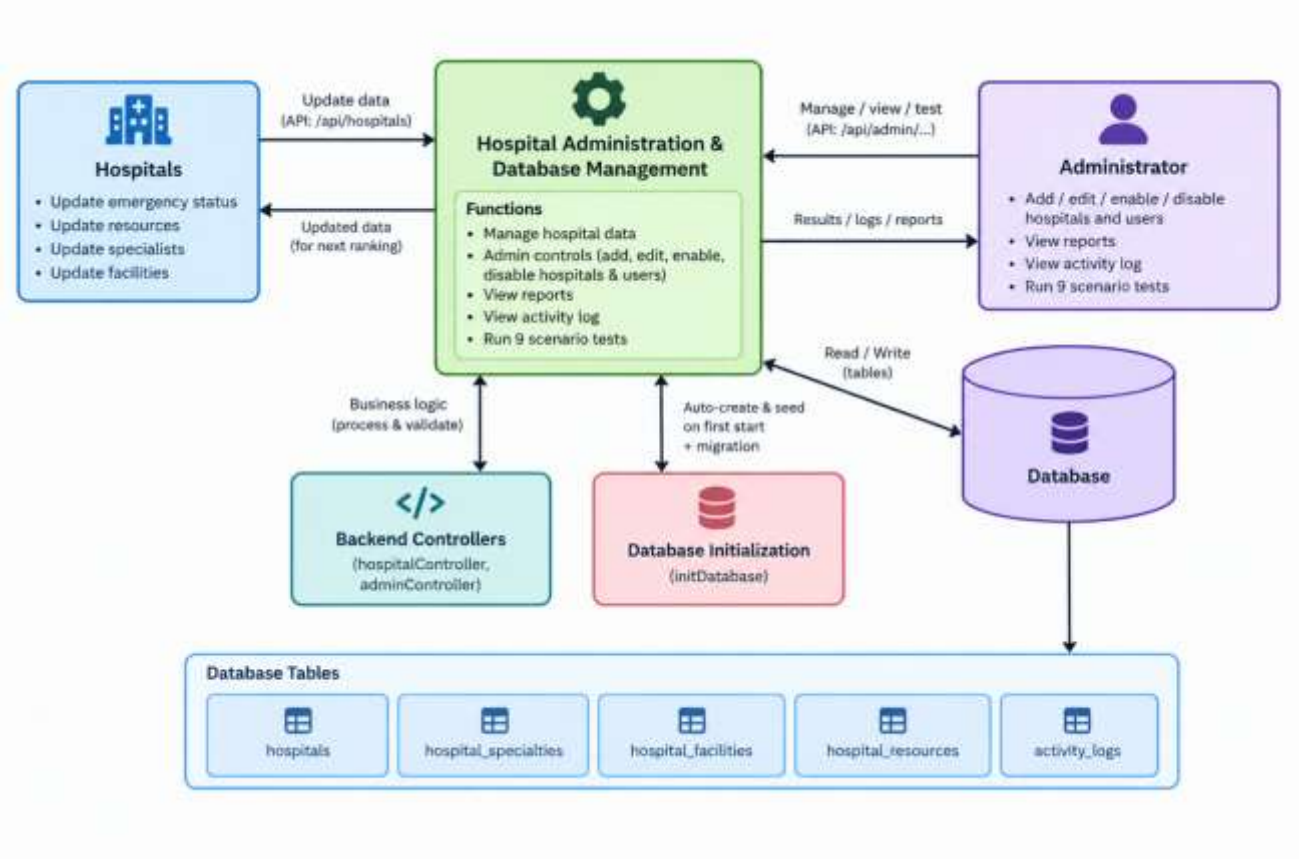


Fig 3.6 DFD for Hospital Administration and Database Management

3.3 DATA FLOW / UML DIAGRAMS

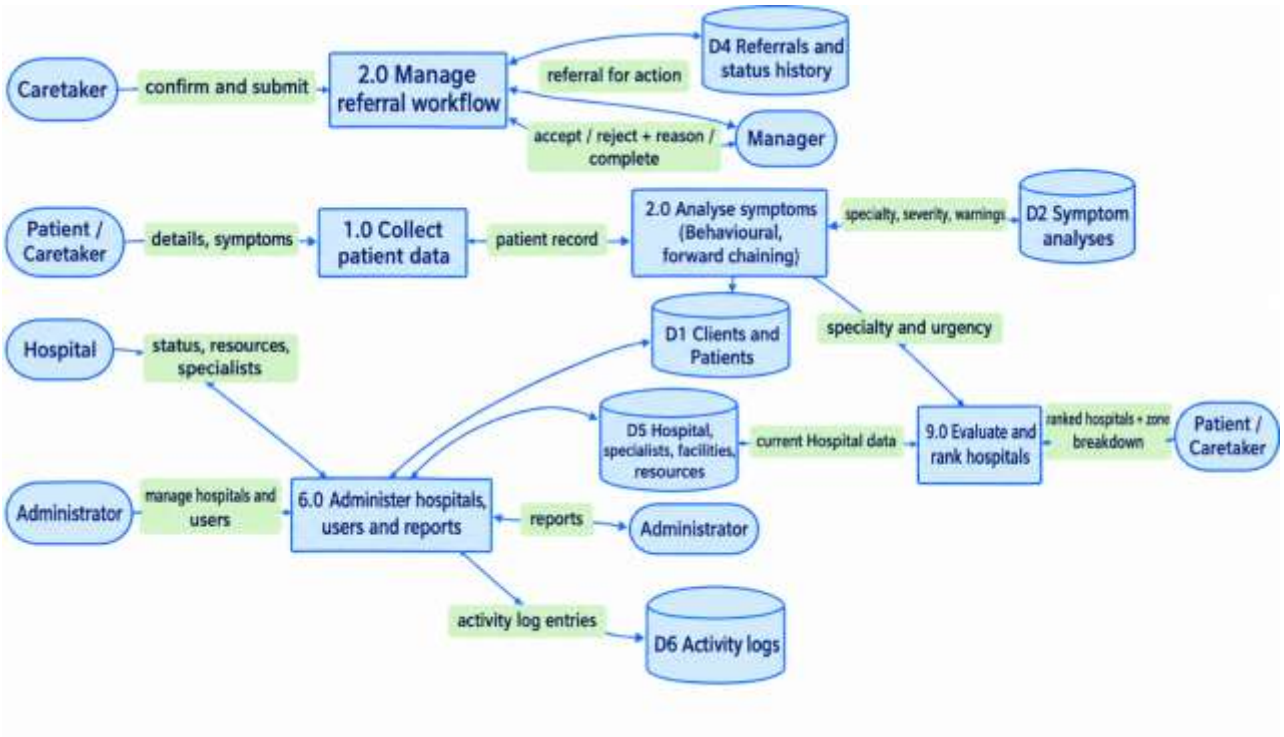


Fig 3.7 Overall Data Flow Diagram (DFD) of the AI Smart Patient Referral and Hospital Selection System

Figure 3.7 illustrates the overall Data Flow Diagram (DFD) of the AI Smart Patient Referral and Hospital Selection System. It shows the interactions between patients, caretakers, hospitals, and administrators, along with the main processes for patient data collection, symptom analysis, hospital evaluation and ranking, referral management, and administration. The diagram also represents how patient details, symptom analysis results, hospital information, referral records, and activity logs are stored and exchanged throughout the system.

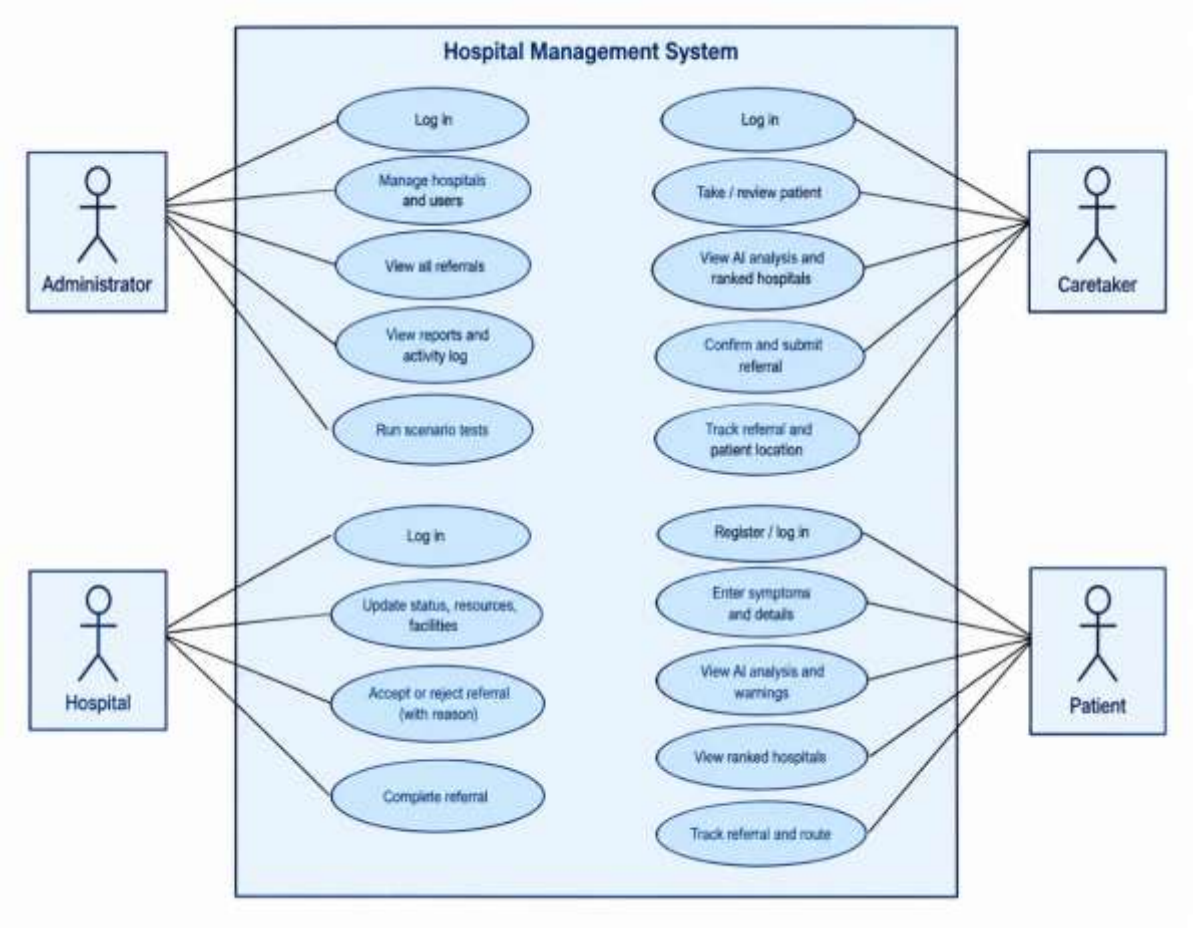


Fig 3.8 Use case diagram

Figure 3.8 illustrates the use case diagram of the AI Smart Patient Referral and Hospital Selection System. It shows the four actors - Patient, Caretaker, Hospital, and Administrator and their interactions with the system, including patient registration, symptom analysis, hospital recommendations, referral management, hospital updates, and administrative tasks.

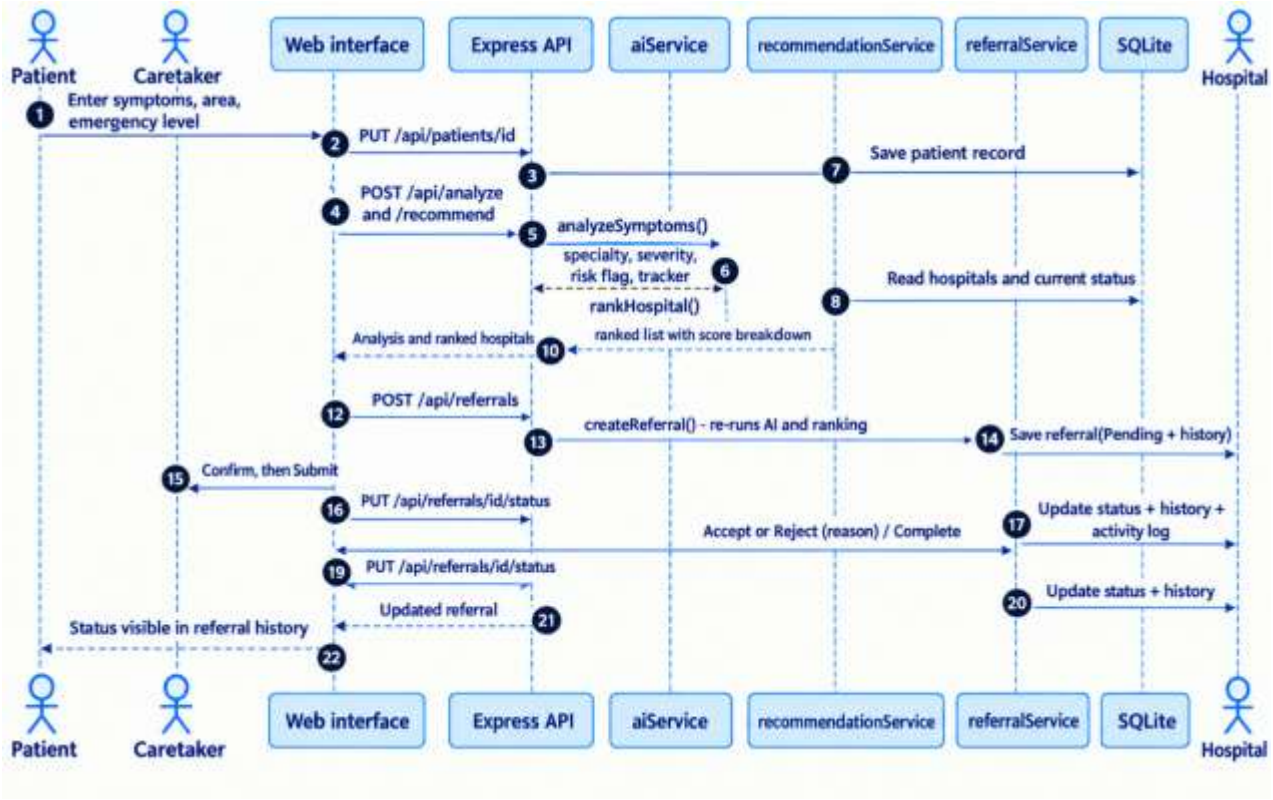


Fig 3.9 Sequence diagram

Figure 3.9 illustrates the sequence diagram of the AI Smart Patient Referral and Hospital Selection System. It shows the step-by-step interaction between the patient, system, AI analysis service, hospital recommendation service, caretaker, and hospital, from symptom entry and hospital ranking to referral creation and the hospital's acceptance or rejection response.

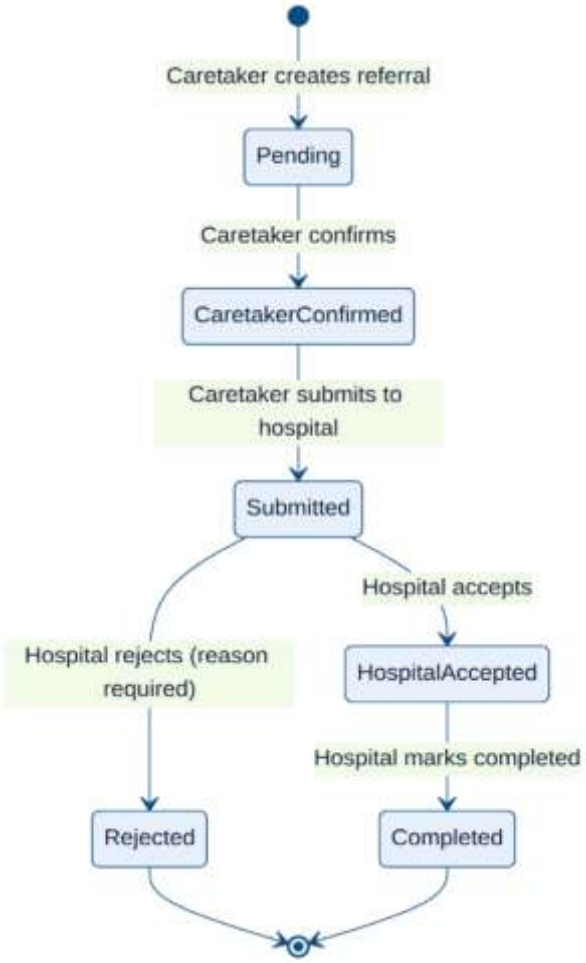


Fig 3.10 Referral state diagram

The referral state diagram illustrates the different stages of the referral process and the permitted transitions between them. A referral begins in the Pending state, moves to Caretaker Confirmed, and is then Submitted for hospital review. The hospital can either Accept or Reject the referral. If accepted, the referral proceeds to the Completed state once the process is finished. This diagram helps ensure that referrals follow a defined workflow and that each status change is tracked systematically.

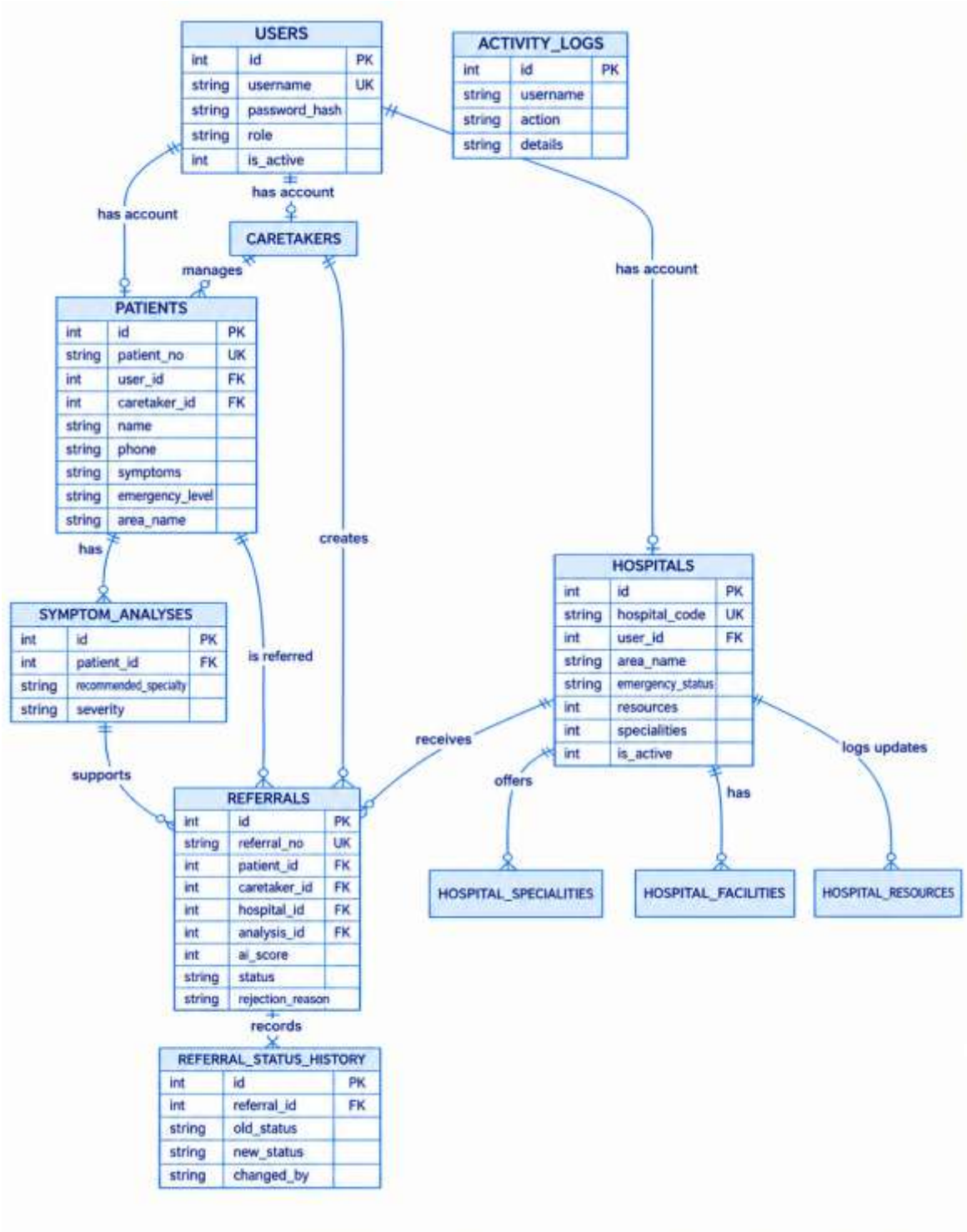


Fig 3.11 Entity-relationship diagram

The entity-relationship diagram illustrates the database structure and the relationships between the main tables in the system. Users are associated with patients, caretakers, and hospitals. A caretaker can manage multiple patients, while each referral connects a patient, caretaker, hospital, and symptom analysis. Each referral can have multiple status-history records to track updates and decisions. The diagram also defines the primary and foreign keys used to maintain data integrity and ensure consistent relationships between the tables.

3.4 THREAT MODEL AND ATTACK SURFACE

This is a semester prototype with synthetic data, so the model is realistic but not exhaustive. The method used is STRIDE.

3.4.1 Assets to protect

- Patient personal and symptom data, and emergency contacts
- User credentials and active sessions
- Hospital availability data, which drives the rankings
- Referral records and their history
- The activity log, as evidence of who did what

3.4.2 Trust boundaries

1. Between the browser and the server (untrusted input crosses here).
2. Between the server and the database file (stored data).
3. Between the browser and third-party services (CDNs, map tiles, routing server).

3.4.3 Attack surface

The attack surface refers to all the parts of the application that could be targeted by attackers. The main entry points and their risks are explained below.

Entry point	Why it is exposed
Login and register endpoints	Open to anyone, so exposed to guessing and abuse
Session cookie	Holds the logged-in identity
REST API (/api/...)	Accepts user-supplied data and IDs
Admin endpoints	High privilege
Free-text fields (symptoms, names, reasons)	Could carry scripts or odd input
Static file server	Serves the front-end files
Third-party scripts, tiles and routing	Outside the project's control
Demo credentials in the seed data	Known passwords if left in place
SQLite file on disk	Holds all data

Table 3.3 System Attack Surface

3.4.4 Threats (STRIDE)

The system addresses security threats using bcrypt password hashing, secure session settings, role-based access control, ownership checks, parameterised SQL queries, and referral transition validation. Activity logs and referral history support accountability, while generic error messages and output escaping help prevent information disclosure and cross-site scripting. However, some risks remain: weak password requirements and the absence of login rate limiting or MFA increase authentication risks; the lack of HTTPS exposes network traffic;

and missing CSRF tokens and CDN integrity checks create additional security concerns. These risks should be addressed before deploying the prototype in a real-world environment.

3.5 SECURITY CONTROLS

3.5.1 Controls Implemented

The following security controls have been implemented in the prototype:

- Authentication: Passwords are stored using bcrypt hashing, sessions are regenerated at login, and generic error messages prevent revealing whether a username exists.
- Session Management: HTTP-only, same-site cookies, an eight-hour session lifetime, and environment-based session secrets are used. Disabled accounts are rejected on subsequent requests.
- Authorisation: Role-based middleware and ownership checks restrict access to authorised users and records.
- Input Validation: Server-side validation checks data types, lengths, and permitted values.
- Injection Protection: Parameterised SQL queries are used throughout the application to prevent SQL injection.
- Output Encoding: User-provided text is escaped before being displayed to reduce cross-site scripting risks.
- Workflow Integrity: Valid referral transitions, ownership checks, duplicate referral prevention, hospital availability checks, rejection reasons, and database constraints maintain data consistency.
- Accountability: Activity logs and referral status history record important actions and status changes.
- Privacy: Patients can access only their own records, registered phone numbers cannot be changed through the update API, and passwords are never returned.
- Error Handling: A central error handler returns structured JSON responses and generic messages for unexpected errors.
- Secrets Management: Secrets are stored in the .env file, while sensitive configuration and database files are excluded from version control.
- Resilience: Offline map fallback, automatic database creation, and database migrations support continued operation.

These controls address access control, auditability, data integrity, authentication, and other security requirements. However, transmission security through HTTPS has not yet been implemented.

3.5.2 Known Weaknesses and Recommended Improvements

The application has several security weaknesses that must be addressed before production deployment. HTTPS should be enabled, and session cookies should use the secure flag to protect data transmitted between the client and server. Cross-Origin Resource Sharing (CORS) should be restricted to trusted origins, and credentialed cross-origin access should be avoided unless necessary. Login rate limiting and temporary account lockouts should be introduced to prevent brute-force attacks, while the password policy should be strengthened by increasing the minimum password length and reviewing the bcrypt cost factor, currently set to 8. CSRF protection should be implemented using tokens or an appropriate alternative, and security headers should be configured using middleware such as Helmet. In addition, in-memory sessions should be replaced with a persistent session store, and Subresource Integrity (SRI) hashes should be added to externally hosted scripts or third-party libraries should be hosted locally. The application should also prevent production startup when the required session secret is missing, implement suitable encryption for stored data, and remove demonstration credentials from production environments in favor of secure credential setup. Finally, multi-factor authentication (MFA) and appropriate email or one-time password (OTP) verification should be introduced to strengthen user authentication and identity verification.

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The system does not train on a dataset. It works from data that users enter, a small rule base, a hospital table and a list of areas, so there is no model-training dataset to collect. All data is synthetic. No real patient or hospital data is used.

4.1.1 Data sources

Source	Content	How it is Obtained
User-entered patient data	Identity, symptoms, conditions, location, emergency level and optional contact details	Registration and Patient Details forms
Hospital Data	Specialties, facilities, specialist count, resource level (0–100), emergency status, beds and contacts	Seed file, then updated by hospital and admin users
Location Data	52 predefined Chennai areas with approximate latitude and longitude	Built-in list in the code, no external lookup
Knowledge Base	5 specialties, 26 symptoms with weights, synonym lists and red-flag rules	Written for this project
System-generated data	Analyses, referrals, status history, activity log	Created as users work

Table 4.1 Data sources of the proposed system

Table 4.1 shows the five data sources used in the proposed system. First, user data includes patient and caretaker details, symptoms, health conditions, location, emergency level, and contact information. Second, hospital data includes specialties, facilities, available beds, resources, emergency status, and contact details. Third, location data contains 52 predefined areas in Chennai with approximate latitude and longitude. Fourth, the knowledge base contains 5 specialties, 26 symptoms, synonyms, and emergency warning rules. Fifth, system-generated data includes analyses, referrals, status history, and activity logs. All data used in the system is synthetic, and no real patient or hospital records are used.

4.1.2 Patient Data Fields

The patient registration form collects the following information, with validation applied where specified:

- Name: 2–60 characters; letters, spaces, dots, apostrophes, and hyphens are allowed.
- Phone: Exactly 10 digits. The registered phone number is read-only after registration.
- Age: Whole number between 0 and 120.
- Gender: Male, Female, or Other.
- Hours of Pain: Numeric value between 0 and 1000.
- Emergency Level: Critical, High, Moderate, or Low.
- Area / Address: Free text up to 120 characters, matched against a predefined area.
- Symptoms: Selected symptoms from a checklist, with optional free text up to 1000 characters.
- Existing Conditions and Medications: Optional text fields.
- Emergency Contact: Contact name, phone number, and relationship, along with notification consent.

4.1.3 Demonstration hospital data

ID	Hospital	Area	Specialties	Facilities	Speciali-sts	Resources
H001	Apollo Medical Center	Anna Nagar	Cardiology, Neurology, Emergency Medicine	ICU, Cath Lab, MRI, CT Scan, 24/7 Emergency	10	95
H002	City General Hospital	T Nagar	General Medicine, Cardiology, Orthopedics	ICU, CT Scan, Emergency, Laboratory	8	88
H003	Sri Lakshmi Multi-Speciality Hospital	Adyar	Neurology, Nephrology, General Medicine	ICU, MRI, Dialysis, Emergency	9	91
H004	Care Plus Hospital	Velachery	Pulmonology, Cardiology, General Medicine	ICU, Ventilator, CT Scan, Emergency	7	84
H005	Green Valley Hospital	Tambaram	Orthopedics, General Medicine, Emergency Medicine	Emergency, X-Ray, Laboratory, Pharmacy	6	78

Table 4.2 Sample hospital data

Table 4.2 shows sample hospital data used in the proposed system. It includes five hospitals in different areas of Chennai, along with their specialties, facilities, number of specialists, and resource levels. This data helps the system identify suitable hospitals based on the patient's needs and the hospital's available resources. All hospital details, including phone numbers, registration numbers, bed counts, ambulance details, working hours, and founding years, are fictional and used only for demonstration purposes.

4.1.4 Storage

Everything is stored in a SQLite database of 12 tables. On first start the system creates the tables and loads the demonstration data: 5 hospitals, 8 caretakers, 12 patient records, the area list and demo accounts. Deleting the database file and restarting rebuilds it.

4.2 DATA PREPROCESSING

Data preprocessing is an important step in the proposed system because it ensures that the patient information is clean, valid, and ready for symptom matching and hospital recommendation. First, input cleaning and validation are performed by removing extra spaces, checking input lengths, and allowing only valid values. Invalid information, such as an age of 200 years, is rejected by the system. Next, text normalisation converts all text into lowercase, removes unnecessary punctuation, and reduces repeated spaces. For example, “Heart Pain!!” becomes “heart pain”.

The system then performs synonym mapping to match different words or phrases with the 26 predefined symptoms. For example, “heart pain” is mapped to “chest pain,” and “breathing problem” is mapped to “breathing difficulty.” Negation handling is also applied to avoid identifying symptoms that the patient has denied. For instance, “no fever” is not counted as a fever symptom. In addition, existing medical conditions are considered when calculating specialty scores. For example, a history of heart disease adds 2 points to Cardiology when the symptoms already indicate that specialty.

Location matching is performed by comparing the user's entered location with the predefined Chennai areas and their alternative names. For example, “Tnagar” is matched with “T Nagar.” If the location is unknown, the system uses Chennai Central as the default and marks the location as approximate. Finally, hospital data is prepared for ranking by considering only active hospitals. Hospital resources, distance, and the number of specialists are converted into ranking scores. For example, a hospital resource score of 95 contributes 19 out of 20 points to the resource component. These preprocessing steps help the system analyse patient information consistently and recommend suitable hospitals based on symptoms, location, and hospital resources.

4.2.1 ALGORITHMS / METHODOLOGY

Algorithm 1: Rule-based symptom analysis (forward chaining)

```
INPUT  : symptomText, conditions, emergencyLevel ∈ {Low, Moderate, High, Critical}
OUTPUT : specialty, severity, confidence, warning,
         detectedSymptoms, scores, reason, trace

1. text ← Normalize(symptomText)          // lowercase, strip noise, collapse spaces
2. facts ← canonical symptoms found in text (direct phrases and synonyms),
3. FOR each specialty DO
     scores[specialty] ← sum of the weights of the facts listed in its rule
4. FOR each condition modifier (condition, specialty, bonus) DO
     IF scores[specialty] > 0 AND condition appears in conditions THEN
       scores[specialty] ← scores[specialty] + bonus     // only where evidence exists
5. IF facts = empty THEN
     specialty ← General Medicine ;  result ← "Insufficient information"
   ELSE
     specialty ← the one with the highest score
     (ties broken in order: Cardiology, Neurology, Pulmonology,
      Orthopedics, General Medicine)
6. redFlags ← matched emergency patterns, such as:
     chest pain + shortness of breath, chest pain + sweating,
     face droop, slurred speech, weakness on one side, low oxygen,
     breathing difficulty + (High or Critical) level
7. IF redFlags not empty THEN  severity ← High
   ELSE IF topScore ≥ 2    THEN  severity ← Moderate
   ELSE                          severity ← Low
   severity ← max(severity, emergencyLevel)   // entered level can raise, never lower
8. confidence ← None if insufficient, else High (≥6) / Medium (≥3) / Low
   RETURN specialty, severity, confidence, warning,
          detectedSymptoms, scores, reason, trace
```

The system uses forward chaining: the detected symptoms are the facts, and the weighted rules lead to the conclusion. It does not name any disease.

Algorithm 2: Hospital scoring and ranking

```
INPUT  : specialty, severity (from Algorithm 1), patientArea, list of hospitals
OUTPUT : ranked list of hospitals, each with score and breakdown

1.  urgent ← true if severity = High OR a red flag is present, else false
2.  FOR each hospital h that is enabled (skip disabled hospitals) DO
      // specialty match (max 35)
      specialtyScore ← 35  if h offers the exact specialty
                       10  if h offers General Medicine (fallback)
                        0  otherwise
      // resources (max 20)
      resourceScore  ← resources / 100 * 20
      // emergency capability (max 15)
      emergencyScore ← min(15, base + 3·[Emergency Medicine] + 3·[ICU])
                       where base = 12 if 24/7 Emergency, 9 if Emergency, else 0
      // distance (max 15) — km from Haversine
      distanceScore  ← max(0, 15 * (1 - km / 30))
      // specialist availability (max 15)
      specialistScore ← min(specialists, 10) / 10 * 15
      // penalty for availability
      penalty ← 40 if Unavailable and urgent    | 25 if Unavailable and routine
                10 if Limited and urgent        |  5 if Limited and routine
                 0 otherwise
      sum   ← specialtyScore + resourceScore + emergencyScore
            + distanceScore + specialistScore
      score[h] ← clamp(sum - penalty, 0, 100)
3.  SORT hospitals by score, highest first   (ties broken by shorter distance)
4.  RETURN the ranked list with each hospital's score breakdown
```

A case counts as urgent when its severity is High or a red flag is present. Referrals to disabled or Unavailable hospitals are blocked.

Algorithm 3: Distance (Haversine formula).

For two points with latitudes φ_1, φ_2 and longitudes λ_1, λ_2:

$a = \sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)$ $d = 2R \cdot \arcsin(\sqrt{a})$, with $R = 6371$ km

This gives the straight-line distance between the centres of two areas. It is not a road distance.

Algorithm 4: Referral workflow (state machine)

A referral moves only along allowed steps: Pending, Caretaker Confirmed, Submitted, then Hospital Accepted or Rejected (a reason is required), then Completed. Each step is checked for the allowed role and ownership, and each change is stored in the status history.

Route display-The map shows the patient's area and the hospital. It requests road route options (fastest, shortest and alternatives) from the OSRM routing service, and falls back to an offline diagram if the internet is unavailable. Routes are for display only. They do not change the ranking.

4.2.2 RESULTS AND DISCUSSION

The results below come from running the system on the demonstration data.

4.2.2.1 Rule-based symptom analysis

The patient entered chest pain, shortness of breath and sweating, with emergency level High and area Egmore. The system detected the three symptoms and scored each specialty. Cardiology scored 7, Pulmonology 2 and General Medicine 1, so the recommended specialty is Cardiology with High severity. A red-flag pattern (chest pain with shortness of breath) triggered the emergency warning. The result lists the detected symptoms, the specialty scores and the reasoning trace.



Fig 4.1 AI analysis result with specialty scores, severity, emergency warning and reasoning trace

4.2.2.2 Hospital ranking with score breakdown



Fig 4.2 Ranked hospitals with score breakdown (all hospitals Available)

4.2.2.3 Route map

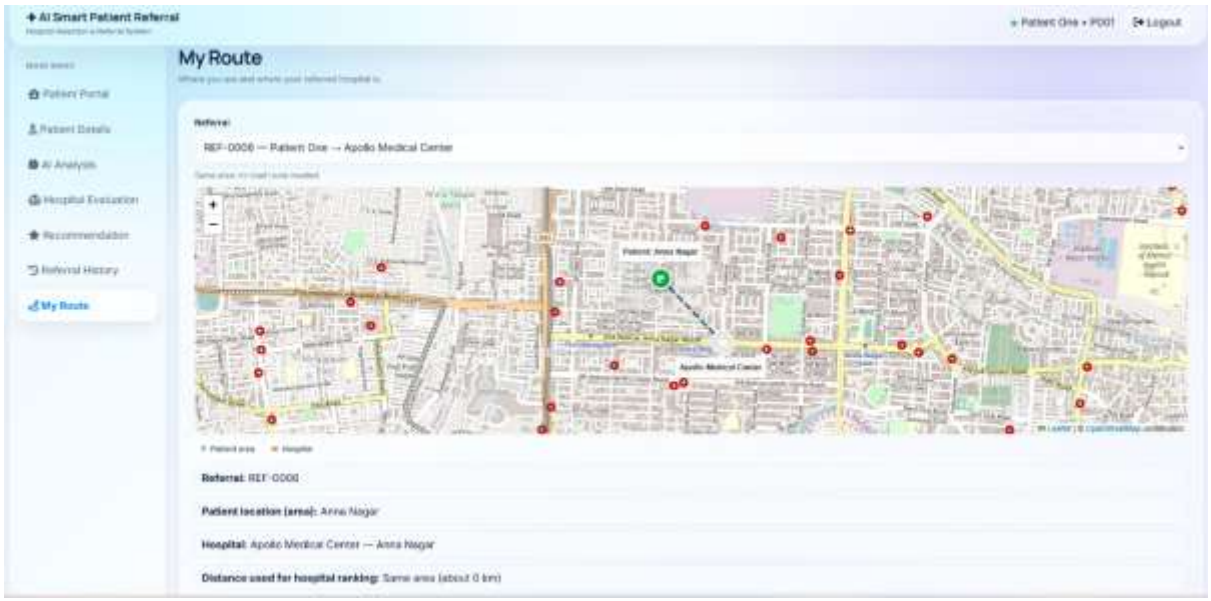


Fig 4.3 Route map with route options from the patient's area to the hospital

4.2.2.4 Scenario test results



Fig 4.4 Scenario test results (9 of 9 passed)

Discussion

- **The rules behave as designed**

  In all five symptom scenarios the expected specialty ranked first, and the three emergency scenarios raised the warning.

- **Explanations are traceable**

  Every result lists the detected symptoms, the contributing rules and the score components, so a reviewer can check why a hospital was ranked where it was.

- **Live availability matters**

  The Unavailable test shows the ranking reacting immediately to a hospital update, which a static directory cannot do.

- **Ties are handled simply**

  The worked input in 4.2 gives Cardiology 4 and Pulmonology 4. The system picks Cardiology by its fixed priority order. This is transparent but a simplification, and showing the second specialty is a possible improvement.

- **The scores are design choices**

  The weights and point values were chosen for this project. They are not learned and they are not clinically validated.

- **Compared with the literature**

  Studies such as [2] report agreement rates for machine-learning triage (53.8%). Those figures are not comparable with the scenario tests here, which check logic and not accuracy.

- **Limits**

  The data is synthetic, there are only 5 hospitals, the symptom vocabulary is small, and distances are straight-line estimates between area centres. A real deployment would need clinical validation and real data.

5.1 CONCLUSION

The AI-Based Smart Patient Referral and Hospital Selection System addresses a practical problem: choosing a suitable hospital for a patient is usually slow, inconsistent and unexplained. The project replaced that guesswork with an explainable process. A rule-based knowledge base of weighted symptom rules, applied by forward chaining, suggests a possible specialty area and an urgency level, with red-flag warnings for emergency patterns. Hospitals are then excluded when disabled or unavailable, and the rest are ranked best first with a transparent score out of 100. That score is built from specialty match, resources, emergency capability, distance and specialist availability. Every result comes with the symptoms that triggered it and a breakdown of the score.

The system goes beyond a recommendation. A caretaker confirms each referral, the hospital accepts or rejects it with a stated reason, and every status change is stored with who made it and when. Hospitals can update their own status and resources, and the next ranking changes immediately. The prototype serves four roles with server-side access control, hashed passwords, validated input, parameterised queries and an activity log. It also includes a route map between the patient's area and the hospital, with an offline fallback.

The nine scenario tests passed on the demonstration data. They confirmed the expected specialty in the five symptom cases, the emergency warnings, the effect of a hospital becoming unavailable (Apollo's score fell from 99 to 59), and the referral workflow. The analysis and ranking logic ran in well under a millisecond on the development machine. These tests check the software's logic and workflow. They do not prove clinical accuracy.

The project has clear limits. It uses synthetic data only and a small rule base of five specialties. Distances used for ranking are straight-line estimates between area centres, and the security is at prototype level. It provides decision support and does not diagnose, and it has not been clinically validated.

5.2 REFERENCES

[1] Chi, C.-L., Street, W. N., & Ward, M. M. (2008). Building a hospital referral expert system with a prediction and optimization-based decision support system algorithm. Journal of Biomedical Informatics, 41(2), 371–386.

[2] Abdel-Hafez, A., Jones, M., Ebrahimabadi, M., Ryan, C., Graham, S., Slee, N., & Whitfield, B. (2023). Artificial intelligence in medical referrals triage based on clinical prioritization criteria. Frontiers in Digital Health, 5, Article 1192975.

[3] Määttä, J., Lindell, R., Hayward, N., Martikainen, S., Honkanen, K., Inkala, M., Hirvonen, P., & Martikainen, T. J. (2023). Diagnostic performance, triage safety, and usability of a clinical decision support system within a university hospital emergency department: Algorithm performance and usability study. JMIR Medical Informatics, 11, e46760.

[4] Omotosho, A., Adegbola, O., & Adebo, A. (2016). A patient-based hospital referral decision support system. International Journal of Computer Applications, 155(10), 38–43.

[5] Semigran, H. L., Linder, J. A., Gidengil, C., & Mehrotra, A. (2015). Evaluation of symptom checkers for self diagnosis and triage: Audit study. BMJ, 351, h3480.

[6] Sutton, R. T., Pincock, D., Baumgart, D. C., Sadowski, D. C., Fedorak, R. N., & Kroeker, K. I. (2020). An overview of clinical decision support systems: Benefits, risks, and strategies for success. *npj Digital Medicine, 3*, Article 17.

[7] Chambers, D., Cantrell, A. J., Johnson, M., Preston, L., Baxter, S. K., Booth, A., & Turner, J. (2019). Digital and online symptom checkers and health assessment/triage services for urgent health problems: Systematic review. *BMJ Open, 9*(8), Article e027743.

[8] Luo, W., & Wang, F. (2003). Measures of spatial accessibility to health care in a GIS environment: Synthesis and a case study in the Chicago region. *Environment and Planning B: Planning and Design, 30*(6), 865–884.

[9] Shortliffe, E. H. (1976). *Computer-based medical consultations: MYCIN.* Elsevier.

[10] Sinnott, R. W. (1984). Virtues of the haversine. *Sky and Telescope, 68*(2), 158–159.

5.3 APPENDIX

Appendix A: Demonstration Accounts (synthetic data, for demonstration only)

Role	Username	Password
Patient	Patient1	Patient123
Caretaker	Caretakere1 to caretaker8	care123, care124, care125, care126, care127, care128, care129, care130
Hospital	hospital1 to hospital5	hospital123, city123, lakshmi123, careplus123, green123
Administrator	admin	Admin123

Table A.1 Demonstration User Accounts

New patients can register with a name, phone number and password. The name becomes the username.

Appendix B: Installation and Running

1. Install Node.js 22 LTS.
2. Open the project folder in a terminal.
3. Run npm install.
4. Run npm start.
5. Open http://localhost:3000 in a browser.

The database is created and filled with demonstration data on the first start. To reset it, stop the server, delete the three smart_referral.db* files in the database folder, and start again. The route map needs internet. Without it, an offline diagram is shown.

Appendix C: Project Structure

```
AI-SMART-REFERRAL/
  frontend/     index.html, script.js, style.css
  backend/      server.js, routes/, controllers/, services/, middleware/, db/
  database/     schema.sql, seed.sql
  package.json, .env.example, README.md
```

The four services are aiService (rule-based analysis), recommendationService (hospital ranking), referralService (workflow) and locationService (areas and distance).

Appendix D: Database Tables

- users – Stores login accounts, user roles, and hashed passwords.
- patients – Stores patient details, symptoms, registered phone numbers, and emergency contacts.
- caretakers – Stores caretaker profiles and related information.

- hospitals – Stores hospital profiles, operational status, available resources, and specialists.
- hospital_specialties – Stores the medical specialties offered by each hospital.
- hospital_facilities – Stores the facilities available at each hospital.
- hospital_resources – Maintains the history of hospital resource availability and status updates.
- symptom_analyses – Stores AI-generated symptom analysis results.
- referrals – Stores referral records, including scores, explanations, and referral status.
- referral_status_history – Records every referral status change, including who made the change and when.
- activity_logs – Records user logins and other important system activities.
- areas – Stores predefined areas in Chennai along with their geographical coordinates.

Appendix E: Hardware and Software Requirements

Hardware	**Requirement**
Processor	Dual-core, 2 GHz or higher
Memory	4 GB (8 GB recommended)
Storage	500 MB free space
Network	Internet for the route map only

Software	**Requirement**
Operating system	Windows 10/11 (macOS or Linux also work)
Runtime	Node.js 22 LTS with npm
Back end	Express 4, SQLite with better-sqlite3, bcryptjs, express-session, dotenv, cors
Front end	HTML5, CSS3, JavaScript, Leaflet 1.9.4 with OpenStreetMap
Browser and tools	Chrome, Edge or Firefox; Visual Studio Code

Table E.1 Hardware & Software Requirements

Appendix F: Additional Screenshots

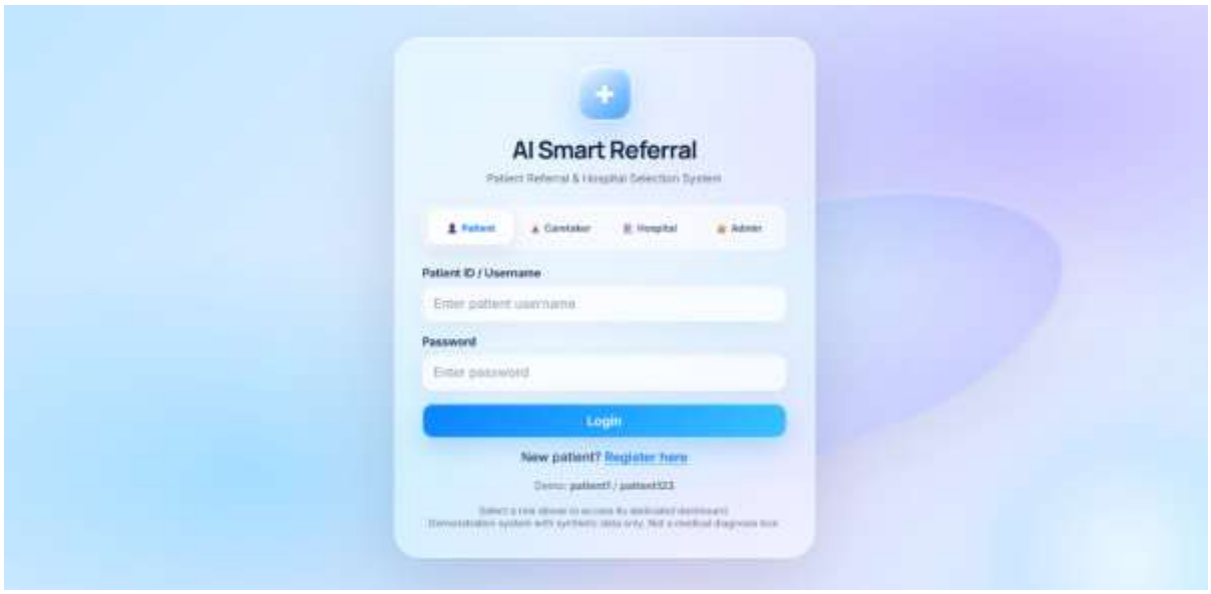


Fig A.1 Login screen with the four role tabs

A simple login page for an AI Smart Referral System with options for patients, caretakers, hospitals, and admins to log in or register.

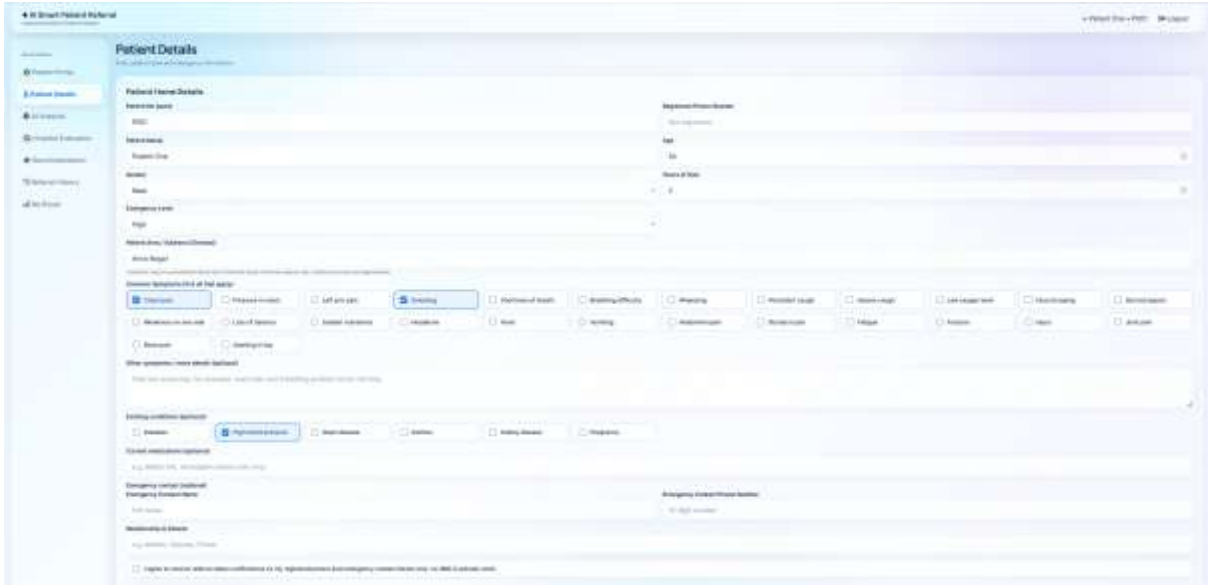


Fig A.2 Patient Details form with the registered phone

A patient details page where users can enter personal information, symptoms, medical conditions, medications, and emergency contact details.

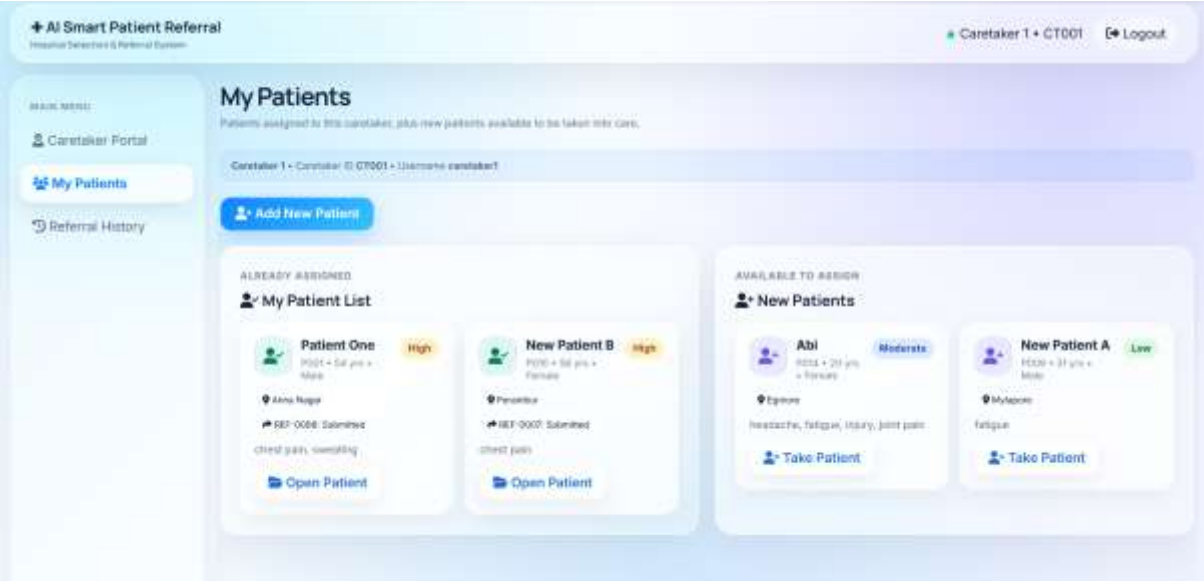


Fig A.3 Caretaker patient list

The My Patients page lists the caretaker's assigned patients, with their symptoms, emergency level and latest referral status, and the new patients waiting to be taken into care. The caretaker can open a patient, take a new patient, or add a new patient from this page.

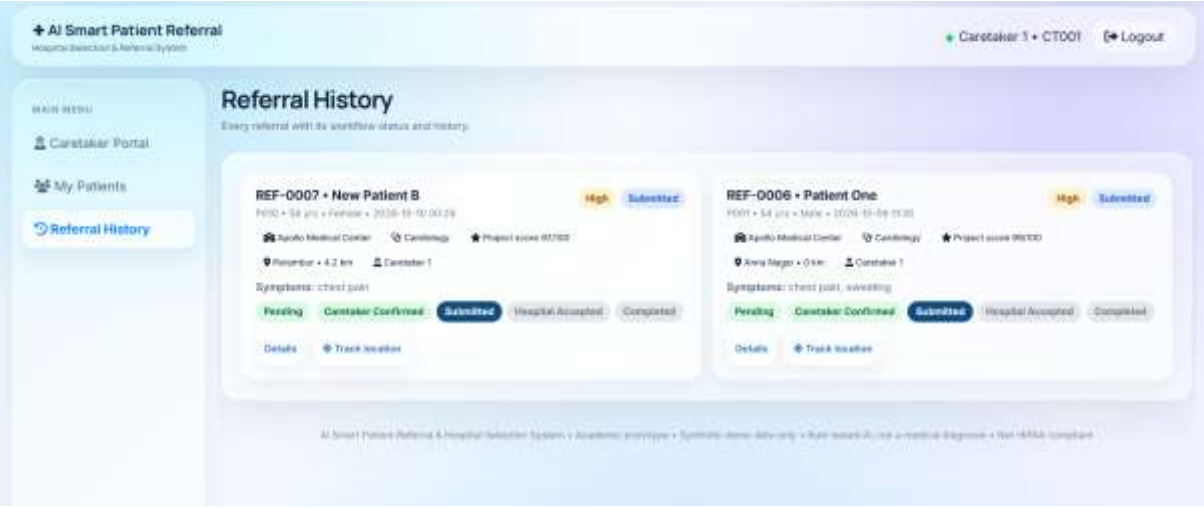


Fig A.4 Referral History with the status steps

The Referral History page lists each referral with its patient, hospital, specialty, project score, distance and symptoms. A step tracker shows the current status (here Submitted), and the Details and Track location buttons open the status history and the map.

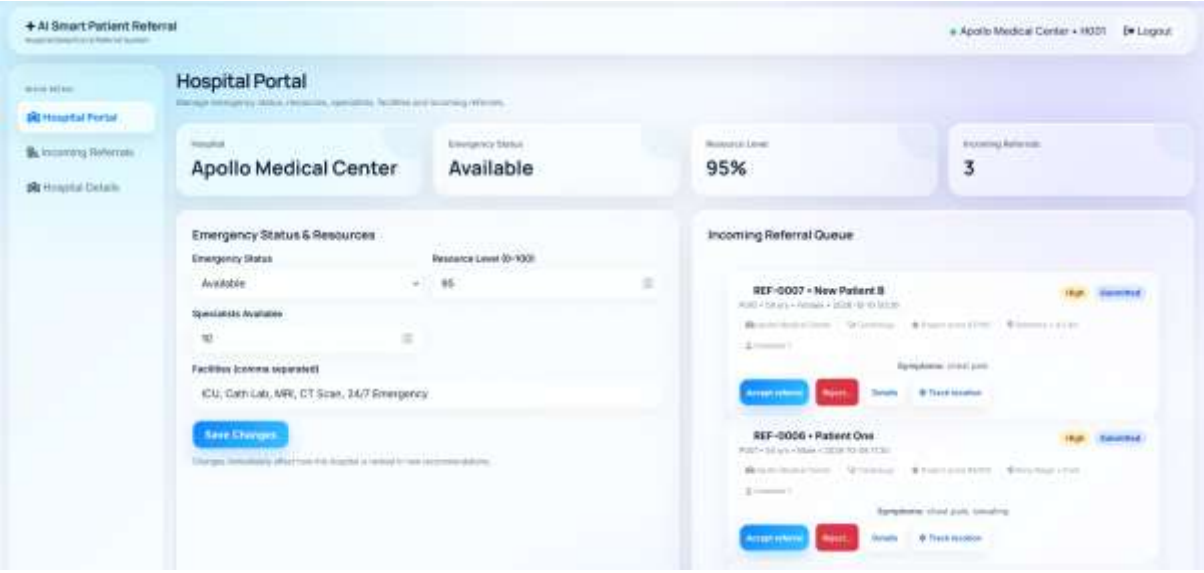


Fig A.5 Hospital dashboard with accept and reject buttons

The Hospital Portal lets a hospital update its emergency status, resources, specialists and facilities, and shows its incoming referral queue. Each submitted referral can be accepted or rejected, or opened for details and location.

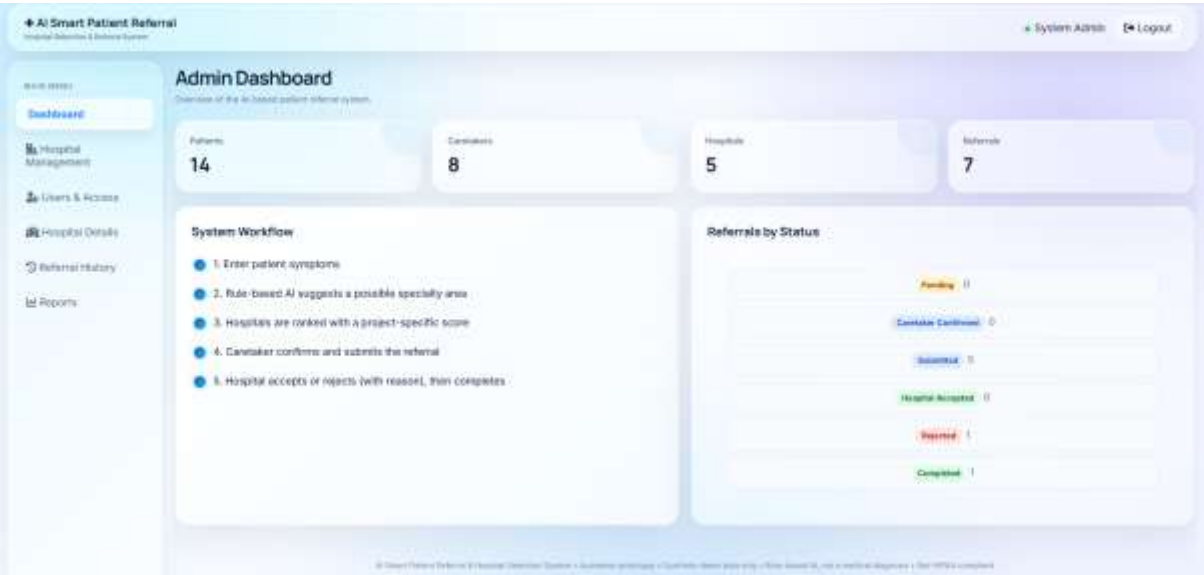


Fig A.6 Admin dashboard

The Admin Dashboard shows the totals of patients, caretakers, hospitals and referrals, the system workflow, and the number of referrals in each status.

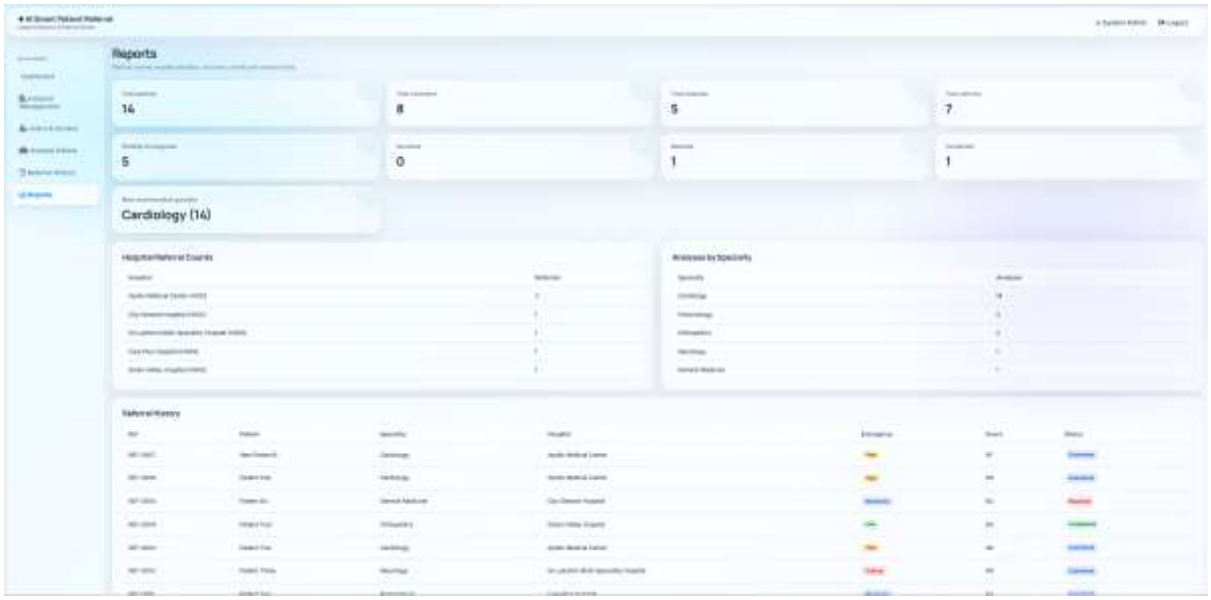


Fig A.7 Admin reports

The Reports page presents the figures behind the dashboard in more detail. Cards show total patients, caretakers, hospitals and referrals, then the number of pending, accepted, rejected and completed referrals, and the most recommended specialty (Cardiology). Below them, tables show the referral count for each hospital, the number of analyses for each specialty, and the full referral history with patient, specialty, hospital, emergency level, score and status.